AI 시대의 소프트웨어 개발

알아야 지시하고, 알아야 검증한다

검토본

마르코 지음

AI 시대의 소프트웨어 개발: 알아야 지시하고, 알아야 검증한다

검토본
[1판 1쇄 펴낸 날 자리]

지은이 마르코
펴낸이 [자리]
펴낸곳 [출판사 자리]
출판 등록 [등록 번호와 날짜 자리]
주소 [자리]
전화 [자리] · 팩스 [자리]
홈페이지 [자리] · 전자우편 [자리]

ISBN [종이책 ISBN 자리]
ISBN [전자책 ISBN 자리]
값 [자리]

© 2026 마르코

이 책은 저작권법에 따라 보호받는 저작물이므로 무단 전재와 무단 복제를 금합니다. 이 책 내용의 전부 또는 일부를 이용하려면 저작권자와 [출판사 자리]의 서면 동의를 받아야 합니다.

[잘못 만든 책의 교환 안내: 출판사 문안 자리]

이 책에 나오는 회사명과 제품명은 각 회사의 상표 또는 등록 상표입니다. Java는 Oracle, Spring은 Broadcom, Claude는 Anthropic, GitHub는 GitHub, Docker는 Docker의 상표이며, 그 밖의 이름도 각 소유자의 것입니다. 본문에서는 ™, ® 표시를 생략했습니다. 이 책은 언급한 회사와 제휴하거나 후원을 받지 않았고, 각 회사가 이 책을 검토하거나 보증하지 않았습니다. `CLAUDE.md`, `AGENTS.md` 같은 파일 이름은 도구가 읽는 파일을 설명하려고 그대로 적었습니다.

책 속 AI의 답은 이런 일이 생길 수 있다는 것을 보여 주려고 다듬은 예시입니다. 같은 요청을 넣어도 도구와 시점에 따라 답은 달라질 수 있습니다.

종이책의 글꼴은 본문 Noto Serif KR, 제목과 상자와 표 Pretendard, 코드 D2Coding이며, 세 글꼴 모두 SIL Open Font License 1.1에 따라 썼습니다. 전자책은 읽는 기기의 글꼴을 따릅니다.

예제 저장소의 코드는 MIT 라이선스로 공개합니다. 누구나 자유롭게 쓰고, 고치고, 다시 배포할 수 있습니다.

머리말

앞 권 『웹 애플리케이션은 어떻게 동작하는가』를 쓰며 저는 프레임워크가 대신해 주는 일을 먼저 직접 해 보자고 권했습니다. 소켓으로 HTTP 서버를 만들고, servlet과 JSP로 화면을 올리고, JDBC로 글을 저장한 뒤 그 모두를 Spring MVC로 옮겼습니다. 대신해 주는 일이 무엇인지 알아야 잘못되었을 때 어디를 볼지 안다는 믿음이었습니다.

그 사이 도구는 한 걸음 더 나아갔습니다. 이제는 "블로그 만들어 줘" 한 줄이면 AI가 Spring Boot 프로젝트를 통째로 만들어 줍니다. 앞 권에서 몇 주 걸린 블로그가 10분 만에 실행됩니다. 그렇다면 앞 권에서 익힌 것은 쓸모가 줄었을까요? 저는 반대라고 생각합니다. AI가 만든 코드는 그럴듯합니다. 그럴듯하게 맞는 코드와 그럴듯하게 틀린 코드를 가려내려면, 그 코드가 무엇을 하는지 읽을 수 있어야 합니다.

그런데 읽을 수 있는 것만으로는 부족했습니다. AI가 만든 블로그에는 아무도 정하지 않은 것이 가득했습니다. 목록은 어떤 순서로 보여야 하는지, 쓰다 만 글은 누가 볼 수 있는지, 서버를 끄면 글은 어떻게 되는지. 코드를 아무리 읽어도 이 질문에는 답이 없습니다. 정하지 않았기 때문입니다. AI는 정하지 않은 자리를 가장 그럴듯한 값으로 채우고, 우리는 그 값이 우리가 원한 것인지 확인하지 않은 채 넘어가기 쉽습니다.

이 책은 그 틈을 다룹니다. 원하는 것을 스스로 찾아내고, 오해 없이 말하고, 만들어진 것이 원한 것인지 확인하고, 요구가 바뀌어도 무너지지 않게 만드는 일입니다. 부제 "알아야 지시하고, 알아야 검증한다"는 앞 권과 이 책을 잇는 한 문장입니다. 동작하는 원리를 알아야 무엇을 확인할지 알고, 무엇을 원하는지 알아야 무엇을 지시할지 압니다.

이 책도 두원의 블로그 이야기로 진행됩니다. 앞 권에서 블로그를 직접 만든 두원이 이번에는 같은 블로그를 AI에게 맡기고, 블로그가 친구들의 블로그에서 낯선 독자가 몰려오는 서비스로 자라는 동안 부딪히는 일을 하나씩 겪습니다. 테스트가 모두 초록불인데 알림이 두 번 가고,

정리 스크립트 한 줄에 운영 데이터가 사라집니다. 그때마다 두원은 AI에게 “고쳐 줘”라고 하기 전에 무엇이 정해져 있지 않았는지부터 찾습니다.

AI 도구는 이 책이 나온 뒤에도 빠르게 바뀔 것입니다. 그래서 특정 도구의 사용법보다, 도구가 바뀌어도 사람이 채워야 하는 칸이 어디인지를 보이려고 했습니다. 앞 권을 읽지 않았어도 이 책을 읽을 수 있습니다. 앞 권의 내용이 필요한 자리에서는 한두 문장으로 다시 설명하고, 어느 장에서 직접 만들었는지 적어 두었습니다.

2026년 10월 27일

마르코

차례

머리말 3

이 책의 구성 15

이 책을 읽는 법 19

일러두기 23

프롤로그: AI는 시킨 대로 만든다. 원하는 대로 만드는 건 사람의 몫이다 25

1부 시킨 대로 만든다 29

1장 "블로그 만들어 줘": 10분 만에 생긴 블로그 31

1.1 한 줄로 시작하기 33

1.2 따라 하기: 빈 폴더에서 첫 커밋까지 35

1.3 AI가 만든 코드 들여다보기 40

1.4 AI가 잘한 것 44

1.5 직접 시험해 보기: 어디가 위험한가 44

1.6 아무도 정하지 않은 것들 51

1.7 개발자의 일은 어디로 갔나 53

2장 "임시저장도 넣어 줘" 했더니 미완성 글이 목록에 나왔다 59

2.1 두 가지 요청 60

2.2 AI가 바꾼 코드 61

2.3 무엇이 잘못되었나 64
2.4 AI는 틀리지 않았다 67
2.5 그냥 "고쳐 줘"라고 하면? 68
2.6 AI가 바꾼 것을 읽고 나누기 70
2.7 되돌리고, 브랜치에서 시도하기 73
2.8 개발자의 일이 바뀐다 80

2부 원하는 것을 정확히 말하기 87

3장 내가 뭘 원하는지 모를 때: AI에게 나를 인터뷰시키기 89

3.1 코드 대신 질문을 요청하기 91
3.2 인터뷰 92
3.3 결정 기록 96
3.4 좋은 질문에는 모양이 있다 101
3.5 인터뷰의 함정 102
3.6 내 결정의 약점을 묻기 103
3.7 인터뷰가 끝나면 104

4장 어디까지 한 번에 맡길까: 쪼개서 말하기 113

4.1 한 덩어리 블로그 115
4.2 맡기는 단위가 곧 확인하는 단위다 116
4.3 겉을 먼저 정하고, 속을 맡긴다: 인터페이스와 저장소 패턴 117
4.4 옮기기만 하는 요청: 리팩터링 121
4.5 맡기기 좋은 단위의 조건: 응집도와 결합도 123
4.6 나눈 다음의 요청은 이렇게 달라진다 124
4.7 직접 하던 순서를 도구로: 명세 도구 126

5장 "API 만들어 줘" 했더니 /getPost와 /deletePost가 생겼다: 함수에서 API까지의 작업 지시서 137

5.1 작업 지시서의 네 칸 139
5.2 명세 없이, 명세와 함께 139
5.3 실패도 명세하라: 예외 설계 143

5.4 "API 만들어 줘" 146
5.5 API도 작업 지시서다: REST 설계 146
5.6 OpenAPI로 겉을 먼저 정하기 153
5.7 계약서는 생겼는데 161

6장 "발행 시각 없는 발행 글"이 생겼다: 지켜야 할 규칙 167

6.1 상태를 바꾸는 방법이 여러 개일 때 169
6.2 글이 언제나 지켜야 할 규칙: 불변식 170
6.3 규칙을 글 안에 가두기: 캡슐화 174
6.4 setter를 걷어 내는 순서 178
6.5 규칙이 어긋나는 순간을 잡는다: 빨리 실패하기 183
6.6 AI에게 규칙을 알려 주기 185
6.7 v2, 그리고 남은 질문 188

3부 안전망 193

7장 명세를 테스트로: 사람이 테스트를, AI가 구현을 195

7.1 직접 하던 확인을 코드로 196
7.2 명세 한 줄, 테스트 하나 199
7.3 AI에게 테스트까지 맡기면 생기는 일 203
7.4 순서를 뒤집는다 207
7.5 AI에게 테스트를 맡겨도 되는 때 211
7.6 한 시간이 1초로 214

8장 기계가 먼저 보는 품질: 린트와 SonarQube 223

8.1 테스트는 통과하는데 읽기 힘든 코드 225
8.2 포매터: 겉모양은 기계가 맞춘다 227
8.3 린트: 흔한 실수는 기계가 잡는다 229
8.4 SonarQube: 코드의 건강 검진 232
8.5 품질 게이트: 기준을 정하고 못 넘으면 막는다 235
8.6 분석 결과를 다시 AI에게 238
8.7 기계가 못 보는 것 243

8.8 품질 대시보드를 시작한다 244

4부 바뀌어도 무너지지 않는 구조 249

9장 "댓글 넣어 줘" 했더니 서비스가 800줄이 됐다: 단일 책임 원칙 251

9.1 한 클래스에 무엇이 들어 있나 253
9.2 바뀌는 이유가 하나인 클래스: 단일 책임 원칙 255
9.3 나눈 대로 지시한다 256
9.4 너무 잘게 나누면 262
9.5 목록을 그대로 넘겨주면 생기는 일 264
9.6 필요한 만큼만 꺼내 주기: 페이지네이션 268
9.7 품질 대시보드, v3 273

10장 DB로 바꿨더니 테스트는 통과, 실제로는 실패했다: 리스코프 치환 원칙과 의존성 역전 원칙 279

10.1 "DB로 바꿔 줘" 281
10.2 같은 인터페이스, 다른 약속 283
10.3 갈아끼워도 믿을 수 있어야 한다: 리스코프 치환 원칙 285
10.4 객체의 규칙을 지키며 저장하기 292
10.5 표는 누가 정했나 294
10.6 서비스가 데이터베이스를 모르게: 의존성 역전 원칙 301
10.7 반쯤 옮겨진 글: transaction 307
10.8 v4를 마치며 311

11장 기존 코드를 건드리지 않고 기능 늘리기: 개방-폐쇄 원칙과 인터페이스 분리 원칙 319

11.1 넣을 때마다 고치는 코드 321
11.2 바뀔 부분과 바뀌지 않을 부분 322
11.3 새 기능은 새 파일로: 개방-폐쇄 원칙 327
11.4 실패에도 종류가 있다 332
11.5 확장 지점은 두 번째에 만든다 335
11.6 뚱뚱한 인터페이스: 인터페이스 분리 원칙 336
11.7 확장 지점은 공격 지점이 되기도 한다 338
11.8 v5를 마치며 343

12장 원칙으로 AI를 지시하고 리뷰하기 353

12.1 초록불이 말해 주지 않는 것 355
12.2 작업 지시서: 원칙을 빈칸으로 바꾸기 356
12.3 리뷰 질문: 원칙을 질문으로 바꾸기 360
12.4 합친 뒤에 나온 버그 367
12.5 로그가 아무 말도 하지 않았다 374
12.6 4부를 마치며 383

5부 혼자에서 팀으로 389

13장 AI 시대의 애자일 (1): 스프린트를 계획하고 나눠 맡기기 391

13.1 만드는 게 빨라지면 무엇이 느려지나 392
13.2 무엇부터: 백로그와 스프린트 목표 394
13.3 스토리와 수용 기준 395
13.4 작은 PR로 나눠 맡기기 398
13.5 누구인지, 무엇을 해도 되는지: 로그인과 권한 400
13.6 충돌을 AI에게 풀게 했더니 410
13.7 검색은 또 하나의 목록이다 413
13.8 지금은 괜찮지만, 언젠가는 느려진다 417

14장 AI 시대의 애자일 (2): 스프린트를 끝내고 돌아보기 423

14.1 작성자가 떠날 때 425
14.2 "끝났다"의 뜻: 완료의 정의 432
14.3 API를 바꾸면 앱이 깨진다: 하위 호환성 434
14.4 회고: 일하는 방식도 고친다 443

15장 다른 사람이 이어받는 블로그 451

15.1 무엇이 코드에 있고, 무엇이 머릿속에 있나 453
15.2 사람용 입구: README를 시험한다 454
15.3 AI용 입구: context 문서 455
15.4 문서보다 검사 461

15.5 부탁이 아니라 장치로: 하네스 465
15.6 확인에서 배포까지 한 흐름으로: CI/CD 468
15.7 붙여 넣기 대신 연결하기 480
15.8 두원 없이 합쳐진 첫 PR 483

6부 진짜 사용자를 만나다 489

16장 "사진도 넣게 해 줘" 했더니 배포할 때마다 사진이 사라졌다: 파일 업로드와 저장소 추상화 491

16.1 "글에 사진도 넣게 해 줘" 493
16.2 사흘 만에 생긴 네 가지 사고 493
16.3 바깥에서 온 이름이 안쪽 경로가 됐다: 경로 조작 495
16.4 사진을 어디에 두는지 모르게 497
16.5 이름표가 아니라 내용을 본다: 파일 시그니처 501
16.6 한번 나간 주소는 약속이다 507
16.7 예서의 화면에서 난 CORS 오류 509
16.8 v8을 마치며 515

17장 "빠르게 해 줘" 했더니 내린 글이 계속 보였다: 성능 측정, 캐시, 무중단 배포 521

17.1 "빠르게 해 줘" 523
17.2 고치기 전에 잰다 524
17.3 AI에게 실행 계획을 보여 준다 526
17.4 캐시도 약속을 지켜야 한다: 캐시 어사이드 530
17.5 서버가 두 대가 되자 536
17.6 멈추지 않고 배포하기: 롤링 배포 542
17.7 잴 수 있으면 지켜볼 수 있다 545

18장 "예약 발행해 줘" 했더니 알림이 두 번 갔다: 시간, 동시성, transaction 551

18.1 "예약 발행해 줘" 553
18.2 일주일 동안 생긴 일 554
18.3 한 칸에 두 가지 뜻 555
18.4 새 상태를 정식으로 더한다 556
18.5 시간은 바깥에서 넣는다 561

18.6 서버 두 대, 같은 1분: 낙관적 잠금 562
18.7 고쳤는데 또 두 통 567
18.8 두 번 와도 한 번처럼: 멱등성 570
18.9 쪼개 고친 것을 한 장의 지시서로 574
18.10 동시에 부르는 테스트 576

19장 GitHub로 로그인했더니 남의 계정이 열렸다: 소셜 로그인, 외부 API, 보안 583
19.1 한 시간 만에 붙은 GitHub 로그인 584
19.2 GitHub가 알려 주는 것 586
19.3 무엇을 믿을지 먼저 정한다 588
19.4 표준 흐름은 Spring Security에, 남은 연결 코드는 AI에게 591
19.5 GitHub가 느려진 날: 시간 제한과 서킷 브레이커 593
19.6 AI 리뷰가 찾은 우회 경로 596
19.7 안전한 기본값, 그리고 점검을 일과로 600

7부 오래 운영하고 이어 가기 611

20장 "블로그가 안 열려요": 장애 대응과 회고 613
20.1 새벽 3시 47분 615
20.2 원인보다 복구 먼저 617
20.3 AI에게는 읽기만 열어 준다: 최소 권한 620
20.4 무엇이 연결을 붙잡고 있었나 621
20.5 증상 말고 범위를 고치게 625
20.6 누가 아니라 무엇이: 비난 없는 회고 626
20.7 런북: 새벽 4시의 나에게 쓰는 편지 630
20.8 울리는 경보, 듣지 않는 사람: SLO와 오류 예산 631
20.9 사람이 보지 않는 합치기 635

21장 "오래된 댓글 정리해 줘" 했더니 운영 데이터가 사라졌다: 데이터 지키기 643
21.1 "오래된 댓글 정리해 줘" 645
21.2 먼저 멈추고, 먼저 되살린다 646
21.3 백업은 있었다, 복구해 본 적이 없었다 648

21.4 옮긴 데이터는 따로 확인한다 651
21.5 회고: 한 사람의 명령 하나가 왜 8천 개를 지웠나 652
21.6 데이터를 바꾸는 작업의 원칙 654
21.7 누가, 어디서 운영 데이터에 손댈 수 있나 658
21.8 복구 훈련과 시점 복구 660
21.9 보존 기간: 얼마나 두고, 언제 지우고, 누구에게 돌려주나 663

22장 "동아리 게시판을 합쳐 줘": 남이 만든 오래된 코드 이어받기 671

22.1 다섯 해 묵은 게시판 672
22.2 AI의 설명을 믿어도 될까 674
22.3 "깔끔하게 다시 짜 줘" 678
22.4 옛 코드에서는 지금 동작이 명세다 679
22.5 지금 동작을 테스트로 붙잡는다: 특성 테스트 679
22.6 옛 모델과 새 모델 사이에: 부패 방지 계층 683
22.7 조금씩 갈아끼우기: 교살자 무화과 패턴 686
22.8 데이터를 옮긴다: 글자와 시각과 건수 688
22.9 옛 비밀번호와 문자열 SQL 690

23장 "영어로도 읽게 해 줘": 여러 AI와 함께 일하기 699

23.1 영어판에 필요한 것 701
23.2 토요일 아침에 본 것 702
23.3 왜 어긋났나 703
23.4 나누기 전에 공용 기반부터: 다국어의 결정 704
23.5 나눠 맡기기: worktree, 공용 파일의 주인, 순서 709
23.6 쓰는 AI, 테스트하는 AI, 리뷰하는 AI 713
23.7 비용, 그리고 사람이 멈춰서 확인하는 지점 715
23.8 일요일, 다시 나눠 맡기기 717

24장 v0에서 v15까지: 과정을 돌아보고 설명하기 725

24.1 결과가 아니라 과정을 보여 준다 727
24.2 여정을 숫자로 보여 주기 727
24.3 과정이 보이는 README 731
24.4 결정 하나를 깊게 733

24.5 AI가 한 일과 사람이 한 일 734
24.6 오래 사는 블로그에서 배운 것 736
24.7 숫자가 좋은 브랜치도 틀릴 수 있다 738
24.8 한 장으로 정리한 순서 739
24.9 다시 프롤로그로 742

에필로그: AI가 더 좋아져도 남는 것 747

부록 A AI와 개발해도 알아야 할 열일곱 가지 751

부록 B 작업 지시서와 요청 모음 757

부록 C 리뷰 질문, 완료의 정의, 점검표 777

부록 D 용어집 793

부록 E 예제 저장소 안내 805

이 책의 구성

이 책은 스물네 장과 프롤로그, 에필로그로 이루어져 있고, 장들은 일곱 부로 묶인다. 두원은 한 줄 요청으로 받은 블로그(v0)에서 시작해, 블로그가 부딪히는 문제를 장마다 하나씩 풀어 가며 여러 사람이 쓰고 몇 달을 운영하는 블로그(v15)를 만든다. 앞 장의 결정과 코드가 다음 장의 출발점이므로 차례대로 읽기를 권한다.

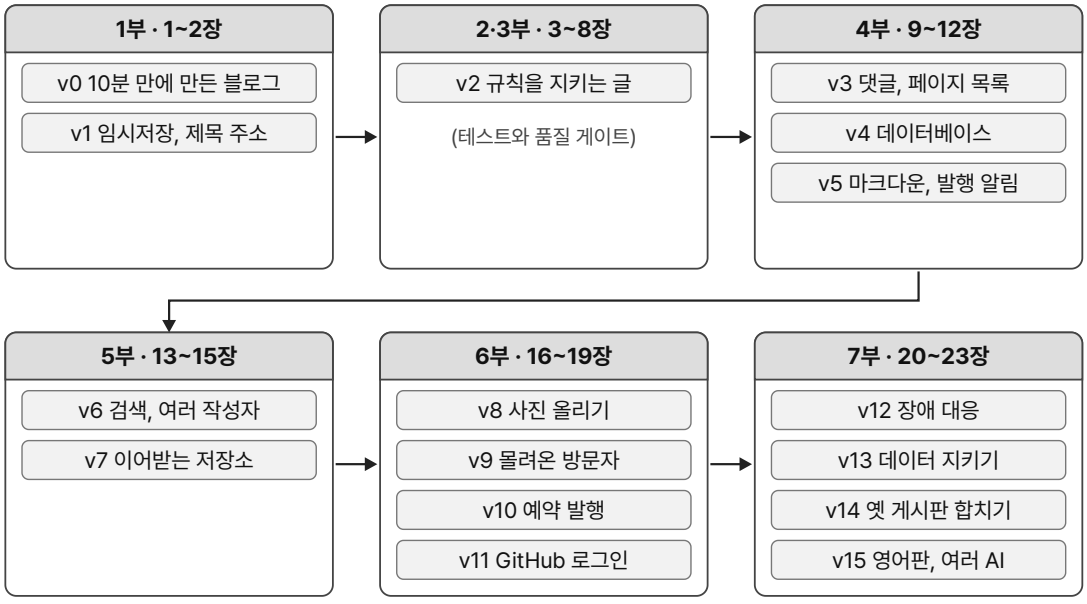


그림 0-1 블로그의 여정 v0 → v15

프롤로그. 앞 권에서 블로그를 직접 만든 두원이 같은 블로그를 AI에게 맡기며 이 책이 시작된다. AI는 시킨 대로 만들고, 원하는 대로 만드는 것은 사람의 몫이라는 이 책의 질문을 세운다.

1부 시킨 대로 만든다

1장 "블로그 만들어 줘": 10분 만에 생긴 블로그. 한 줄 요청으로 Spring Boot 블로그를 받아 실행하고, 앞 권에서 직접 만든 조각(Tomcat, `DispatcherServlet`, 화면, 저장소)이 AI의 코드 어디에 들어 있는지 찾는다. 그리고 코드를 읽어도 답이 없는 것, 곧 아무도 정하지 않은 동작을 표로 모은다.

2장 "임시저장도 넣어 줘" 했더니 미완성 글이 목록에 나왔다. 요청 한 줄에 빠진 결정이 버그가 되는 장면을 겪는다. AI가 바꾼 코드를 diff로 읽고 나눠 커밋하며, 개발자의 일을 정하기, 말하기, 확인하기로 나눈다.

2부 원하는 것을 정확히 말하기

3장 내가 뭘 원하는지 모를 때. AI에게 코드 대신 질문을 시켜 내가 내릴 결정을 찾고, 결정 기록(`docs/decisions.md`)과 명세를 쓴다.

4장 어디까지 한 번에 맡길까. 맡길 단위를 "내가 확인할 수 있는 크기"로 정하고, 겉(약속)을 먼저 정한 뒤 속(구현)을 맡긴다. 명세 도구를 한 바퀴 실행해 사람이 맡을 판단을 가려낸다.

5장 "API 만들어 줘" 했더니 `/getPost`와 `/deletePost`가 생겼다. 입력, 바뀌는 것, 결과, 실패의 네 칸으로 작업 지시서를 쓰고, REST API와 OpenAPI 명세로 넓힌다. 앞 권에서 적은 사전·사후 조건이 지시서의 칸이 된다.

6장 "발행 시각 없는 발행 글"이 생겼다. 어떤 방법으로 바뀌어도 참이어야 하는 불변식을 찾고, AI가 정해진 메서드만 쓰도록 setter를 걷어 낸다.

3부 안전망

7장 명세를 테스트로. 사람이 테스트를 먼저 쓰고 "테스트는 고치지 마"라는 조건으로 AI에게 구현을 맡긴다. 구현을 일부러 비틀어 테스트가 무엇을 지키는지 확인한다.

8장 기계가 먼저 보는 품질. 포매터, 린트, SonarQube 품질 게이트를 GitHub Actions에 추가하고, 경고를 끄는 꼼수를 막는 요청을 쓴다.

4부 바뀌어도 무너지지 않는 구조

9장 "댓글 넣어 줘" 했더니 서비스가 800줄이 됐다. 앞 권에서 이름을 얻은 단일 책임 원칙을, 커밋 기록에서 책임의 경계를 찾아 AI에게 나눠 지시하는 말로 바꾼다.

10장 DB로 바꿨더니 테스트는 통과, 실제로는 실패했다. PostgreSQL로 옮긴 저장소가 약속을 어기는 장면에서 리스코프 치환 원칙과 의존성 역전 원칙을 다시 만나고, 계약 테스트와 패키지 울타리로 AI의 변경을 지킨다.

11장 기존 코드를 건드리지 않고 기능 늘리기. 확장 지점과 작은 인터페이스로 마크다운 본문과 발행 알림을 더하며 개방-폐쇄 원칙과 인터페이스 분리 원칙을 쓴다.

12장 원칙으로 AI를 지시하고 리뷰하기. 다섯 원칙을 작업 지시서의 빈칸과 리뷰 질문으로 바꾸고, 버그를 AI와 함께 좁혀 가는 순서와 다음 버그를 위한 로그를 정한다.

5부 혼자에서 팀으로

13장 AI 시대의 애자일 (1). 정하고 확인하는 속도에 맞춰 스프린트를 계획하고, 수용 기준을 테스트로 바꿔 작은 PR로 나눠 맡긴다. 여러 작성자의 로그인과 검색을 만든다.

14장 AI 시대의 애자일 (2). 떠나는 작성자의 정보, 완료의 정의, 이미 나간 앱을 깨뜨리지 않는 API 변경, 회고를 다룬다.

15장 다른 사람이 이어받는 블로그. 머릿속의 결정을 README, AI context 문서, 자동 검사, 하네스로 옮기고, 배포까지 한 흐름으로 잇는다.

6부 진짜 사용자를 만나다

16장 파일 업로드와 저장소 추상화. 바깥에서 들어온 파일 이름이 경로가 된 업로드 코드를 고치고, 저장 위치를 감춘다. CORS 오류를 "다 허용"으로 덮지 않는다.

17장 성능 측정, 캐시, 무중단 배포. 느린 곳을 재서 찾고, 캐시를 같은 약속을 지키는 구현으로 만들고, 서버 두 대로 멈추지 않고 배포한다.

18장 시간, 동시성, transaction. 예약 발행으로 알림이 두 번 간 원인을 찾고, 충돌 확인과 멱등성으로 막는다. 앞 권에서 겪은 transaction proxy의 함정이 AI의 코드에서 다시 나온다.

19장 소셜 로그인, 외부 API, 보안. GitHub 로그인에서 무엇을 믿을지 정하고, 느린 바깥 API와 악의적인 요청을 명세에 적는다.

7부 오래 운영하고 이어 가기

20장 장애 대응과 회고. 새벽 장애에서 원인보다 복구를 먼저 하고, AI에게는 운영 권한 대신 읽기 전용 창문을 준다. 런북과 서비스 수준 목표를 쓴다.

21장 데이터 지키기. 정리 스크립트 한 줄로 운영 데이터를 잃은 뒤, 미리 보기가 기본인 일괄 작업과 복구 훈련을 만든다.

22장 남이 만든 오래된 코드 이어받기. 앞 권의 두원처럼 Servlet과 JSP로 만든 동아리 게시판을 특성 테스트로 붙잡고, 부패 방지 계층을 두고 조금씩 옮긴다.

23장 여러 AI와 함께 일하기. 영어판을 여러 에이전트에게 나눠 맡기며 작업 폴더, 공용 파일의 주인, 역할, 비용 한도, 사람이 멈출 지점을 정한다.

24장 v0에서 v15까지. 결과가 아니라 과정을 보여 주는 README와 결정 설명서를 쓰고, AI와 개발하는 순서를 한 장으로 정리한다.

에필로그. AI가 더 좋아져도 사람에게 남는 일을 정리한다.

책 끝의 부록 A에는 AI와 개발해도 알아야 할 지식을 왜 알아야 하는지와 책 속 위치를 붙여 모았다. 부록 B에는 작업 지시서와 요청을, 부록 C에는 리뷰 질문과 완료의 정의, 장별 점검표를, 부록 D에는 용어집을, 부록 E에는 예제 저장소의 브랜치와 준비물을 모았다.

이 책을 읽는 법

누가 읽으면 좋은가

AI 도구로 코드를 받아 본 적은 있지만, 받은 코드가 원하는 대로 동작하게 만드는 데서 막히는 개발자를 위해 썼다. 요청 한 줄로 그럴듯한 결과가 나오는데 어디가 틀렸는지 설명하기 어려운 사람, AI가 쓴 테스트가 모두 통과하는데도 마음이 놓이지 않는 사람에게 특히 권한다.

자바로 클래스와 인터페이스를 만들 줄 알고, 웹 애플리케이션이 요청을 받아 응답하는 흐름을 대략 알고 있다고 가정한다. 앞 권 『웹 애플리케이션은 어떻게 동작하는가』가 그 바탕을 다룬다. 앞 권을 읽지 않았어도 따라올 수 있도록, 앞 권의 개념이 필요한 자리에서는 한두 문장으로 다시 설명하고 "앞 권 N장"으로 어디서 직접 만들었는지 적었다. 이 책에서 "N장"은 이 책의 장을, "앞 권 N장"은 앞 권의 장을 가리킨다.

앞 권과 이어지는 곳

이 책에서 다시 만나는 것	앞 권에서 직접 만든 곳
Tomcat, `DispatcherServlet`, 화면과 저장소의 자리(1장)	6~11장
상태 코드, 302와 303, 401과 403(1, 5, 13장)	3, 5, 7, 11장
사전·사후 조건, 계약 테스트(5, 10장)	5, 7장
글 객체가 스스로 규칙을 확인하기, 회귀 테스트(6, 7장)	8장
다섯 가지 설계 원칙(9~12장)	5, 7, 8, 10장
SLF4J와 Logback, 로그 수준(12장)	2장

이 책에서 다시 만나는 것	앞 권에서 직접 만든 곳
thread pool, connection pool(12, 18, 20장)	4, 10장
세션과 쿠키, HttpOnly, XSS(13장)	6장
multipart와 믿지 않는 파일 이름(16장)	8장
데코레이터와 proxy(17, 18장)	3, 9장
transaction, 격리 수준, 자기 호출의 함정(18장)	10장
Servlet과 JSP, `PreparedStatement`(22장)	6, 10장

장의 차례

모든 장은 같은 차례로 구성되어 있다.

1. 장 첫머리의 **이 장에서 배울 것**. 배울 것과 이 장의 실습(저장소, 브랜치, 만들 것)을 적었다.
2. 두원과 마르코가 나오는 여는 장면. 이 장의 문제가 어디서 왔는지 보여 준다.
3. 번호가 붙은 절. 새 요구가 들어오고, AI에게 그냥 맡기고, 결과가 원한 것과 다르다는 걸 알게 되고, 왜 어긋났는지 따져 본 뒤, 정해서 다시 맡기고 확인한다.
4. 장 끝의 **이 장에서 배운 것**(표), **연습 문제**, **마르코의 질문**.
5. 다음 장으로 넘어가는 짧은 장면.

덧붙임 글

이름	무엇을 담나
이 장에서 배울 것	그 장의 학습 목표와 실습(저장소, 브랜치, 만들 것)
같은 기능, 두 가지 요청	그냥 요청한 결과와 정해서 요청한 결과를 나란히 놓은 비교
참고, 팁, 주의	짧은 설명, 알아 두면 편한 것, 실수하기 쉬운 것
개발 상식	그 장의 사건과 이어지는 실무 지식

이름	무엇을 담나
마르코의 질문	개발하며 실제로 마주칠 상황 하나를 마르코가 질문으로 던진다. 짚어야 할 것과 빠지기 쉬운 답을 함께 적었다

블로그가 자랄 때마다 장 끝에 테스트 수와 커버리지 같은 품질 대시보드 숫자를 적었다. 숫자는 자랑하려는 것이 아니라, 숫자가 꺾이거나 움직이지 않는 곳에 무슨 일이 있었는지 보려는 것이다.

책 속 AI의 답

책 속 AI의 답은 이런 일이 생길 수 있다는 것을 보여 주려고 다듬은 예시다. 같은 요청을 넣어도 도구와 시점에 따라 답은 얼마든지 달라진다. 답이 책과 달라도 괜찮다. 눈여겨볼 것은 답의 겉모양이 아니라, 내가 정하지 않은 빈칸이 어떻게 채워졌는가다. 특정 AI 도구의 사용법은 다루지 않는다. 도구가 바뀌어도 사람이 채워야 하는 칸이 어디인지에 집중한다.

예제 저장소와 브랜치

예제 코드는 저장소 `blog-with-ai-source` 하나에 버전별 브랜치(`v0`~`v15`)로 나눠 두었다. 장 첫머리의 "이 장의 실습"에 그 장의 브랜치를 적었고, 브랜치 목록과 준비물, 실행 방법은 부록 E에 모았다. 막히면 그 버전의 코드와 비교해 보자. 브랜치마다 그 시점의 결정 기록(`docs/decisions.md`)이 들어 있으니, 코드를 열기 전에 결정 기록부터 읽으면 그 버전의 코드가 무엇을 지키려 하는지 먼저 알 수 있다.

등장인물

이름	설정
두원	앞 권에서 블로그를 직접 만들었다. 이번에는 같은 블로그를 AI에게 맡기며, 정하고 확인하는 법을 익힌다. 이야기 중간에 인턴십을 거쳐 시간제로 일하게 된다

이름	설정
마르코	두원의 오랜 친구로, 개발을 몇 해 먼저 시작했다. 답을 바로 주기보다 질문을 던지고, 두원이 먼저 부딪혀 보게 둔다
태율	두원과 같은 과 친구. 7장부터 블로그 개발을 함께 하고, 블로그의 두 번째 작성자가 된다
예서	연습으로 블로그 글을 보여 주는 앱을 만든다. 블로그 API의 첫 사용자다
태양	1학년 후배로 Spring은 처음이다. 두원이 인턴십으로 자리를 비운 동안 태율과 함께 블로그를 이어받는다
민찬	두원의 친구. 블로그의 독자로, 이상한 것을 가장 먼저 알려 준다
준서	앞 권에 나온 대학 친구. 블로그의 첫 독자이자 첫 필자다

두원과 마르코, 두원과 친구들은 반말을 쓴다. 태양은 선배들에게 존댓말을 쓰고, 두원은 인턴십 팀의 시니어 개발자에게 존댓말을 쓴다.

실습 환경

도구	쓰는 곳	비고
JDK 21	모든 장	앞 권과 같다
Gradle Wrapper	모든 장	저장소에 든 `./gradlew`를 쓴다. 따로 설치하지 않는다
Git, GitHub	모든 장	2장부터 커밋과 브랜치, 8장부터 GitHub Actions
Docker	8장, 10장부터	SonarQube, PostgreSQL 16, Redis, Testcontainers
AI 코딩 도구	모든 장	특정 도구를 가정하지 않는다. 대화형 AI와 저장소에서 일하는 에이전트를 함께 쓴다
curl	1장부터	Windows는 `curl.exe`

설치와 실행 방법은 부록 E.2와 E.3에 있다. 도구의 버전, 설치 화면, 명령의 출력은 글을 쓰는 시점(2026년 10월)을 기준으로 한다.

일러두기

원어 표기

앞 권의 표기를 따른다. 제품, 라이브러리, 프레임워크, 운영체제의 이름은 원어로 쓴다. Spring, Spring Boot, Gradle, Maven, Tomcat, PostgreSQL, Redis, Docker, GitHub, SonarQube, Windows, macOS가 그렇다. 조사는 원어를 우리말로 읽은 소리에 맞춘다(Gradle을, GitHub와).

기술의 고유한 이름은 업계에서 흔히 쓰는 대소문자를 따른다. Servlet, JSP, JDBC, JSON, JAR, WAR, Javadoc이 그렇다. 흔히 한글로 옮겨 적는 기술 용어 가운데 일부도 앞 권처럼 원어로 쓰되, 일반 명사이므로 소문자로 둔다. annotation, reflection, context, proxy, transaction, thread pool, connection pool, bean, setter, getter, wrapper, coordinates, redirect, escape가 그렇다. 우리말로 굳은 말은 한글로 둔다. 클래스, 메서드, 스레드, 서버, 테스트, 인터페이스, 의존성, 세션, 쿠키, 헤더가 그렇다. 앞 권에서 "규칙"이라고도 부른 것 가운데 어떤 방법으로 바뀌어도 늘 참이어야 하는 것은 이 책에서 **불변식**으로 부른다.

AI와 함께 일하는 데서 새로 생긴 말은 처음 나오는 자리에서 풀어 쓴다. 작업 지시서, 결정 기록, AI context 문서, 하네스, 에이전트가 그렇다. 모아 보려면 부록 D의 용어집을 본다.

글자 모양

모양	쓰는 곳	예
코드 글꼴	클래스, 메서드, 명령, 파일과 폴더 이름, 설정 값. 번역하지 않는다	`PostRepository`, `./gradlew`, `docs/decisions.md`
굵은 글씨	처음 정의하는 낱말	**불변식**
인용 상자	AI에게 보낸 요청과 AI의 답	

테스트 메서드 이름은 업계에서 흔히 하듯 영어로 짓고, 무엇을 확인하는지는 `@DisplayName`에 우리말 문장으로 적는다. 테스트를 실행하면 결과 화면에 그 문장이 나온다.

이야기와 상관없는 코드 부분은 `…`나 `// …`로 줄였다. 줄인 곳을 빼면 코드는 예제 저장소의 것과 같다. 시각, 해시값, 걸린 시간처럼 실행할 때마다 달라지는 값은 독자의 화면과 다를 수 있다.

문장

본문은 "~다" 체로 쓰고, 두원과 마르코의 대화는 친구 사이의 반말로 쓴다. 각주는 쓰지 않는다. 다른 곳을 가리킬 때는 쪽 번호 대신 장과 절의 번호(5.3절), 그림의 번호(그림 3-1)를 쓴다. 앞 권을 가리킬 때는 "앞 권 N장"으로 적는다. 전자책에서는 목차, 절과 그림·예제·표의 번호, 웹 주소를 누르면 그 자리로 이동하고, 코드는 글자 그대로 복사할 수 있다.

그림

그림은 앞 권처럼 먹색 선에 형광펜 세 색(노랑은 지금 보는 곳, 민트는 우리가 쓴 코드, 분홍은 문제나 실패)만 쓰고, 흑백으로 인쇄해도 뜻이 남도록 테두리 모양을 함께 정했다. 화면을 캡처한 그림은 캡션 아래에 출처와 캡처 시기를 적는다.

프롤로그: AI는 시킨 대로 만든다. 원하는 대로 만드는 건 사람의 몫이다

요즘은 “Spring Boot로 간단한 블로그 만들어 줘” 한 줄이면 10분 뒤에 블로그가 돌아간다. 글을 쓰면 목록에 올라오고, 제목을 누르면 본문이 보인다. 코드도 깔끔하다. 그 화면 앞에서 누구나 한 번쯤 같은 생각을 한다. “이제 개발은 AI가 하는구나. 나는 뭘 배워야 하지?”

이 책은 그 질문에 대한 답이다.

AI는 점점 더 잘 만든다

먼저 인정하고 시작하자. AI는 코드를 잘 쓴다. 그리고 앞으로 더 잘 쓸 것이다. 이름 짓기, 클래스 나누기, 테스트 붙이기 같은 일은 시키지 않아도 해 주는 날이 곧 온다. “AI가 만든 코드는 엉망이니 사람이 고쳐야 한다”는 이야기를 기대했다면, 이 책은 그 이야기를 하지 않는다.

대신 이 책은 AI가 아무리 좋아져도 사라지지 않는 일을 다룬다.

말한 것과 원한 것 사이

이 책의 주인공 두원도 블로그를 AI에게 맡긴다. 며칠 뒤 “임시저장 기능도 넣어 줘”라고 요청하자, AI는 성실하게 임시저장을 만든다. 그런데 쓰다 만 글이 목록에 그대로 올라온다. AI가 틀렸을까? 아니다. 두원은 “임시저장 글은 목록에 보이면 안 된다”고 말한 적이 없다. 사실 그런 생각을 해 본 적도 없다.

AI는 내가 **말한 것**을 만든다. 내가 **원한 것**을 만들지는 않는다. 그 사이의 틈은 AI의 성능이 좋아질수록 오히려 더 잘 드러난다. AI가 빠르고 성실하게 만들수록, 내가 무엇을 원하는지 정하지 않은 대가도 빠르게 돌아온다.

이 책은 그 틈을 메우는 방법을 다룬다.

- 내가 무엇을 원하는지 **스스로 찾아내는 법**
- 그것을 AI가 오해하지 않게 **정확히 말하는 법**
- 만들어진 결과가 정말 원한 것인지 **확인하는 법**
- 요구가 바뀌어도 **무너지지 않게 만드는 법**

바탕을 아는 사람이 AI를 만날 때

이 책의 주인공 두원은 앞 권 『웹 애플리케이션은 어떻게 동작하는가』에서 블로그를 직접 만들었다. 자바 파일 하나를 JAR로 묶는 데서 시작해 로그를 남기고, 소켓으로 HTTP 서버를 만들고, Servlet과 JSP로 화면을 올리고, JDBC로 글을 데이터베이스에 저장했다. 마지막에는 그 블로그를 Spring MVC로 옮기며, 직접 만든 조각마다 Spring의 어느 부분이 그 일을 대신하는지 짝지어 보았다. 앞 권의 끝에서 두원은 요청 하나가 소켓과 스레드를 지나 자기 코드에 닿았다가 응답으로 돌아오는 과정을 열 줄로 적을 수 있게 되었다.

사실 두원의 블로그는 AI가 한 줄로 만들어 준 블로그에서 시작했다. 앞 권 첫머리에서 두원은 그 블로그를 마르코에게 자랑했지만, 목록이 어떤 순서로 보이는지, 글이 어디에 저장되는지 답하지 못했다. 앞 권을 마친 두원은 그때의 한 줄을 AI에게 다시 맡겨 본다. 직접 만드는 데 두 달 가까이 걸린 블로그가 이번에도 10분 만에 실행된다. 이번에는 다르다. 두원은 AI가 만든 코드를 열고, 앞 권에서 익힌 대로 이 일을 누가 대신해 주는지부터 찾는다. Spring Boot 안에 Tomcat이 들어 있다는 것, `DispatcherServlet`을 대신 등록해 준다는 것은 금방 읽힌다. 그런데 금방 읽히지 않는 것이 있다. 목록은 어떤 순서로 보여야 하는지, 없는 글을 찾으면 무엇을 돌려줘야 하는지, 서버를 끄면 글은 어떻게 되는지. 코드를 아무리 읽어도 이 질문에는 답이 없다. 아무도 정하지 않았기 때문이다.

이 책은 그 지점에서 시작한다. 바탕을 아는 것은 AI와 일하는 데 꼭 필요하다. 무엇을 봐야 하는지 알아야 결과를 확인할 수 있기 때문이다. 하지만 바탕을 아는 것만으로는 충분하지 않다. 무엇을 만들지 정하고, 그것을 오해 없이 말하고, 만들어진 것이 정말 원한 것인지

확인하는 일이 남는다. 이 책의 부제 "알아야 지시하고, 알아야 검증한다"는 그 두 가지를 함께 가리킨다.

앞 권을 읽지 않았어도 이 책을 읽을 수 있다. 앞 권에서 다룬 개념이 나오면 그 자리에서 한두 문장으로 다시 설명하고, 어느 장에서 직접 만들었는지 적어 두었다. 더 깊이 알고 싶을 때 앞 권의 그 장을 펼쳐 보면 된다.

이 책의 흐름

이 책은 계획 없이 시작한다. 설계도, 요구 사항 문서도 없이 AI에게 "블로그 만들어 줘" 한 줄을 던지는 데서 출발한다. 대부분의 사람이 실제로 그렇게 시작하기 때문이다. 그리고 그렇게 만든 블로그가 부딪히는 문제를 하나씩 겪으며, 그때마다 필요한 지식과 방법을 꺼내 해결해 간다. 원칙을 먼저 배우고 적용하는 순서가 아니라, 문제를 먼저 겪고 그 문제를 푸는 도구로 원칙을 만나는 순서다. 앞 권에서 이미 이름을 얻은 원칙은 다시 가르치지 않는다. 그 원칙을 AI에게 지시하는 말과 결과를 평가하는 질문으로 바꿔 쓴다.

이 책은 처음부터 끝까지 블로그 하나를 이어서 만든다. 한 줄 프롬프트로 만든 블로그(v0)는 임시저장과 규칙을 갖추고, 팀이 이어받는 블로그(v7)를 거쳐, 진짜 사용자를 만나고 몇 달을 운영하는 블로그(v15)가 된다. 장을 거듭하는 동안 매번 같은 일이 벌어진다. 그림 0-1이 그 리듬이다.

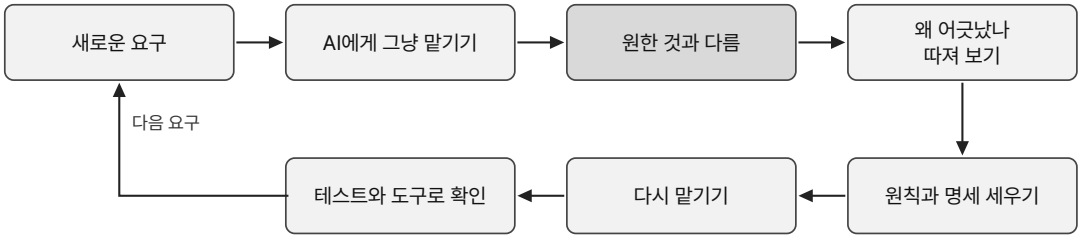


그림 0-1 모든 장의 리듬

새로운 요구가 들어오고, AI에게 그냥 맡기고, 결과가 원한 것과 다르다는 걸 알게 되고, 왜 어긋났는지 따져 본다. 그다음 원칙과 명세(spec)를 세우고, 다시 맡기고, 테스트와 도구로 확인한다. 이 리듬이 몸에 익으면, 어떤 AI 도구를 쓰든 원하는 결과에 가까이 갈 수 있다.

앞 권에서처럼 장마다 두원과 마르코가 등장한다. 마르코는 답을 알려 주기보다 질문을 던지고, 두원이 먼저 부딪혀 보게 둔다. 그 토요일 오후에도 그랬다.

1부

시킨 대로 만든다

한 줄 요청으로 받은 블로그를 실행하고, 앞 권에서 직접 만든 조각이 AI의 코드 어디에 있는지 찾는다. 그리고 요청에 없던 결정이 버그가 되는 장면을 겪는다.

1장 10분 만에 생긴 블로그 · 2장 목록에 나온 미완성 글

1
장

"블로그 만들어 줘"

10분 만에 생긴 블로그

1.1 한 줄로 시작하기

1.2 따라 하기: 빈 폴더에서 첫 커밋까지

1.3 AI가 만든 코드 들여다보기

1.4 AI가 잘한 것

1.5 직접 시험해 보기: 어디가 위험한가

1.6 아무도 정하지 않은 것들

1.7 개발자의 일은 어디로 갔나

이 장에서 배울 것

- AI가 만든 Spring Boot 프로젝트를 실행하고, 앞 권에서 직접 만든 조각이 어디에 들어 있는지 찾을 수 있다.
- 화면과 `curl -i`로 AI가 만든 블로그를 시험하고, 상태 코드로 무엇이 위험한지 말할 수 있다.
- AI가 고친 의존성 설정을 먼저 읽고, 새 라이브러리가 정말 필요한지 확인할 수 있다.
- 요청에 없던 결정이 코드에서 어떻게 채워지는지 찾아 표로 정리할 수 있다.

이 장의 실습

- **저장소:** `blog-with-ai-source`
- **브랜치:** `v0`
- **만들 것:** 한 줄 요청으로 만든 블로그 v0(글쓰기, 목록, 상세 보기), 첫 실행과 첫 커밋, GitHub 원격 저장소

토요일 오후, 두원의 노트북에서는 지난달에 Spring MVC로 옮긴 블로그가 돌아가고 있었다. 두원은 그 뒤로 글을 두 편 더 올렸다. 마르코가 커피 두 잔을 내려놓으며 화면을 들여다봤다.

> **마르코** 잘 버티고 있네. 이거 다 만드는 데 얼마나 걸렸지?
>
> **두원** 소켓부터 치면… 두 달 가까이? 주말마다 했으니까.
>
> **마르코** 그럼 이제 그 블로그를 열어 볼 차례네. 처음 국숫집에서 보여 줬던 거. AI가 10분 만에 만들어 줬다던.
>
> **두원** 어젯밤에 열어 봤어. 그런데 그때 아무렇게나 시킨 거라 폴더도 어수선하고, 커밋은 하나도 없더라. 아예 빈 폴더에서 그때 그 한 줄로 다시 시켜 볼까 해.
>
> **마르코** 좋아. 대신 아무것도 정하지 말고, 그때처럼 딱 한 줄만. 다 되면 같이 열어 보자.

한 시간쯤 지나 두원이 노트북을 돌렸다.

> **두원** 이것 좀 봐. 이번에도 10분 만에 됐어. 글쓰기, 목록, 상세 보기까지 다 돼. 코드도 깔끔하고.

마르코 잘 실행되네. 그럼 국숫집에서 물었던 거 다시 물어볼게. 이 블로그 목록은 어떤 순서로 보여?

두원 직접 만든 블로그라면 바로 답하지. 내가 `ORDER BY id DESC`로 정했으니까. 그런데 이건… 최신 글이 위에 오겠지?

마르코 그렇게 정했어?

두원 아니. 이번에도 정한 적은 없어.

마르코 그럼 지금 이 블로그는 누가 정한 순서로 보여 주고 있을까?

· · ·

1.1 한 줄로 시작하기

두원은 이번에 일부러 아무것도 정하지 않았다. 설계도도 요구 사항 문서도 없이 한 줄만 넣었다. 처음 무언가를 만들 때 대부분의 사람이 실제로 이렇게 시작한다. 이 책도 여기서 출발한다.

두원은 빈 폴더 `blog`를 하나 만들고, 그 폴더에서 AI 도구를 열었다. 입력한 프롬프트는 한 줄이었다.

Spring Boot로 간단한 블로그 만들어 줘. 글쓰기, 목록, 상세 보기가 있으면 돼.

AI는 프로젝트 뼈대를 만들고, 글을 나타내는 클래스와 화면 세 개, 그리고 요청을 처리하는 컨트롤러를 만들었다. 서버를 실행하고 브라우저에서 `/posts`에 들어가면 글 목록이 나오고, "새 글" 버튼을 누르면 글쓰기 화면이 나온다.

일을 마친 AI는 이렇게 보고했다.

Spring Boot 블로그를 만들었습니다.

- `Post`: 글 번호, 제목, 본문, 작성 시각을 담는 레코드
- `PostController`: 목록(`GET /posts`), 상세(`GET /posts/{id}`), 글쓰기 화면(`GET /posts/new`), 저장(`POST /posts`)

- 화면: `list.html`, `detail.html`, `form.html`

간단하게 만들기 위해 데이터베이스 대신 메모리에 저장합니다. `./mvnw spring-boot:run`으로 실행한 뒤 http://localhost:8080/posts 에 접속하세요.

보고는 친절했다. "메모리에 저장합니다"라는 말도 들어 있었다. 다만 그 말이 "서버를 끄면 글이 모두 사라진다"는 뜻이라는 것은 적혀 있지 않았다. 앞 권에서 글을 JSON 파일에, 다시 데이터베이스에 옮겨 담느라 고생한 두원도 그때는 그 한 줄을 그냥 지나쳤다. 대화 창에서는 그림 1-1처럼 보였고, 브라우저로 연 블로그는 그림 1-2와 같았다.



그림 1-1 한 줄 프롬프트와 AI의 응답



그림 1-2 v0 블로그 화면

도구마다 코드의 모양은 조금씩 다르겠지만, 같은 문장을 넣으면 비슷한 블로그가 금방 만들어질 것이다. 이 책의 예제 저장소 `v0` 브랜치에도 같은 블로그가 들어 있다. 다만 비슷하다는 것과 똑같다는 것은 다르다. 그 차이는 1.6절에서 보게 된다.

1.2 따라 하기: 빈 폴더에서 첫 커밋까지

두원이 10분 만에 한 일을 독자도 자기 컴퓨터에서 해 보자. 이 장의 나머지는 이 블로그를 직접 실행해 놓고 읽어야 잘 보인다.

준비물

필요한 것은 네 가지다. 자바 개발 도구(JDK) 21, Git, 코드를 읽을 편집기, 그리고 프로젝트 폴더의 파일을 읽고 고칠 수 있는 AI 도구다. 터미널에서 앞의 둘부터 확인한다.

터미널

```
$ java -version    # 첫 줄에 21이 보여야 한다. 예: openjdk version "21.0.4"
$ git --version    # git version 2.x 가 나오면 된다
```

`java -version`이 17이나 11로 나오면, JDK 21을 설치한 뒤 터미널을 새로 연다. 컴퓨터에 자바가 여러 개 설치되어 있으면 예전 것이 먼저 선택되는 일이 흔하다. 이럴 때 AI에게 "빌드가

안 돼"라고만 말하면, AI는 코드를 고치기 시작한다. 고칠 곳은 코드가 아니라 내 컴퓨터다. AI에게 오류를 물을 때는 `java -version`의 출력도 함께 붙이자.

이 넷 말고도 챙길 것이 셋 있다. 먼저 이 장 끝에서 코드를 올릴 GitHub 계정이다. GitHub는 push할 때 계정 비밀번호를 받지 않는다. 개인 액세스 토큰을 만들어 비밀번호 자리에 넣거나, SSH 키를 등록해 둔다.

Windows에서는 이 책의 명령을 Git Bash(Git과 함께 설치된다)나 WSL(Windows 안에서 리눅스를 실행하는 기능) 같은 bash 셸에서 입력한다. PowerShell에서는 줄 끝의 `\`나 `for` 같은 문법이 다르게 동작한다. 마지막으로 8장부터는 Docker를 쓴다. 설치는 그때 해도 된다.

AI 도구는 프로젝트 폴더에서 연다. 요즘 AI 도구는 연 폴더 아래의 파일을 읽고, 고치고, 명령을 실행할 수 있다. 홈 폴더 전체에서 열면 AI가 볼 수 있는 범위도 홈 폴더 전체가 된다. 맡길 일과 상관없는 파일까지 AI의 손이 닿는 곳에 둘 이유가 없다. 블로그 폴더 하나에서만 연다.

뼈대는 누가 만드나

두원은 빈 폴더에서 AI에게 뼈대까지 맡겼다. AI는 빌드 설정 파일 `pom.xml`과 빌드 도구 실행 파일까지 만들어 두었다. 나중에 보니 그 파일에 적힌 Spring Boot 버전은 그때 나와 있던 버전보다 한참 오래된 것이었다. AI는 지금 나와 있는 버전을 찾아보고 적는 것이 아니라, 학습할 때 알던 버전을 기억으로 적는다.

그래서 이 책은 뼈대를 Spring Initializr(start.spring.io)에서 만들고, 그 폴더에서 AI에게 한 줄을 요청하는 방법을 권한다. 고를 것은 다음과 같다.

표 1-1 Spring Initializr에서 고를 값

항목	고를 값	이유
Project	Maven	앞 권과 같은 빌드 도구다. 이 책의 명령(`./mvnw`)과 설정 파일(`pom.xml`) 모양이 이것을 따른다
Language	Java	
Spring Boot	기본으로 골라진 정식 버전	이름에 SNAPSHOT이나 M이 붙은 것은 시험판이다

항목	고를 값	이유
Group / Artifact	`blog` / `blog`	패키지 이름이 `blog`가 된다
Java	21	설치한 JDK와 맞춘다
Dependencies	Spring Web, Thymeleaf	웹 요청 처리와 HTML 화면

패키지는 4장부터 `blog.post`처럼 기능별로 나눈다. 내려받은 압축 파일을 풀면 설정 파일이 `src/main/resources/application.properties`로 들어 있다. 이 책은 설정을 YAML(들여쓰기로 계층을 나타내는 설정 파일 형식)로 쓰므로 이름을 `application.yml`로 바꾸고, 안에 든 한 줄(`spring.application.name=blog`)을 YAML 모양으로 옮긴다.

예제 1-1 `application.yml`: YAML로 옮긴 설정

```yaml
spring:
  application:
    name: blog
```

그 폴더에서 AI 도구를 연 뒤, 1.1절의 한 줄을 그대로 넣는다. 뼈대가 이미 있으니 AI는 그 위에 파일을 더한다. 혹시 AI가 `pom.xml`의 버전을 바꾸려 하면 멈추게 한다. 1.5절에서 이 파일을 다시 확인한다.

실행해 보기

AI가 일을 마쳤다고 하면 직접 실행한다.

터미널

```
$ ./mvnw spring-boot:run        # Windows에서는 mvnw.cmd spring-boot:run
```

처음에는 필요한 라이브러리를 내려받느라 1~2분 걸린다. 로그 끝에 `Tomcat started on port 8080`과 `Started BlogApplication`이 보이면 서버가 시작된 것이다. 브라우저에서 `http://localhost:8080/posts`를 연다. 끌 때는 터미널에서 `Ctrl+C`를 누른다.

```
… TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
… BlogApplication : Started BlogApplication in 2.695 seconds (…)
```

로그에 Tomcat이 보인다. 앞 권 12장에서 두원은 블로그를 WAR로 묶어 따로 내려받은 Tomcat에 올렸다. Spring Boot는 반대로 Tomcat을 애플리케이션 안에 넣어 함께 실행한다(내장 Tomcat). 그래서 Tomcat을 따로 설치하지 않아도 되고, 빌드한 JAR 파일 하나를 `java -jar`로 실행해도 똑같이 시작된다. 웹 서버와 Servlet container는 어떻게 다른지, 그 앞에 nginx 같은 proxy를 왜 두는지는 15장에서 다룬다.

`Port 8080 was already in use`가 보이면 다른 프로그램이 이미 8080번을 쓰고 있다는 뜻이다. 앞 권의 블로그가 아직 실행 중일 수도 있다. 그 프로그램을 종료하거나, `application.yml`에 `server.port: 8081`을 적고 다시 실행한다.

테스트를 실행해 보기

화면이 나오면 테스트도 한번 실행해 본다.

터미널

```
$ ./mvnw test
```

두원의 터미널에는 이렇게 나왔다.

```
[INFO] --- surefire:3.5.6:test (default-test) @ blog ---
[INFO] No tests to run.
…
[INFO] BUILD SUCCESS
```

`BUILD SUCCESS`라는 초록 글자가 보이지만 그 위에 `No tests to run.`이 함께 찍혀 있다. 실행할 테스트가 하나도 없었다는 뜻이다. AI는 테스트를 만들지 않았고, 두원도 요청하지 않았다. 앞 권에서 계약 테스트와 회귀 테스트를 직접 쓰던 두원에게는 낯선 화면이었다. 성공은 "확인한 것이 모두 맞았다"는 뜻이지, 무언가를 확인했다는 뜻은 아니다.

Spring Initializr로 뼈대를 만들었다면 `BlogApplicationTests`라는 테스트가 하나 들어 있을 것이다. 안에는 `contextLoads()`라는 빈 메서드 하나뿐이다. 이 테스트는 Spring이 오류 없이 시작되는지만 본다. 목록의 순서도, 빈 제목도 확인하지 않는다. 그래도 지우지 말고 두자. 설정을 잘못 건드려 서버가 시작되지 않게 되면 이 테스트가 가장 먼저 알려 준다. 블로그가 원한 대로 동작하는지 확인하는 테스트는 7장에서 쓴다.

첫 커밋

마지막으로 지금 상태를 Git에 남긴다. 이 커밋이 2장부터 AI가 바꾸는 모든 것의 출발점이 된다.

터미널

```
$ git init                # Spring Initializr 프로젝트에는 .gitignore가 이미 있다
  # target/ 이 들어 있는지 본다 (없으면 한 줄을 적어 만든다)
$ cat .gitignore
$ git status              # 올라갈 파일 목록을 확인한다
$ git add .
$ git commit -m "v0: 한 줄 요청으로 만든 블로그"
$ git log --oneline
```

`git status`를 볼 때는 두 가지를 확인한다. `target/` 같은 빌드 결과물이 목록에 없어야 한다. 명령 한 줄로 다시 만들 수 있으니 기록할 필요가 없다. 그리고 비밀번호나 키가 든 파일이 없어야 한다. 한번 커밋한 비밀값은 나중에 지워도 기록에 남는다. 지금 블로그에는 비밀값이 없지만, 커밋 전에 목록을 훑는 습관은 지금부터 들이자.

`git log --oneline`에 커밋 한 줄이 보이면 준비가 끝났다. 이제 AI가 무엇을 바꾸든 이 상태로 돌아올 수 있다.

원격 저장소에 올리기

내 컴퓨터에만 있는 기록은 컴퓨터와 함께 사라진다. GitHub에 빈 저장소를 하나 만든다. README나 `.gitignore`를 더하는 선택지는 고르지 않는다. 내 컴퓨터에 이미 있어서 겹친다. 그다음 저장소 화면에 나오는 주소로 연결한다.

터미널

```
  # 기본 브랜치 이름을 main으로 맞춘다 (이 책은 main을 쓴다)
$ git branch -M main
$ git remote add origin https://github.com/내아이디/blog.git
$ git push -u origin main  # -u는 앞으로 git push만 입력해도 이곳으로 가게 한다
```

push할 때 아이디와 비밀번호를 물으면 비밀번호 자리에 준비물에서 만든 토큰을 넣는다. SSH 키를 등록했다면 주소를 `git@github.com:내아이디/blog.git`으로 쓴다. GitHub 화면에서 파일 목록과 커밋 한 줄이 보이면 된다.

1.3 AI가 만든 코드 들여다보기

AI가 만든 파일은 이렇다. 빌드 도구가 쓰는 파일은 뺐다.

```
blog/
  pom.xml
  src/main/java/blog/
    BlogApplication.java          시작점 (main 메서드)
    Post.java                     글 하나
    PostController.java           요청 처리와 저장
  src/main/resources/
    application.yml               설정 (지금은 거의 비어 있다)
    templates/posts/
      list.html                   목록 화면
      detail.html                 상세 화면
      form.html                   글쓰기 화면
```

자바 파일은 세 개뿐이다. 핵심은 두 파일이다. 먼저 글을 나타내는 `Post`다.

예제 1-2 `Post.java`: 글을 나타내는 레코드

```
public record Post(Long id, String title, String content,
        LocalDateTime createdAt) {
}
```

자바의 레코드(record)로 글 번호, 제목, 본문, 작성 시각을 담았다. 레코드는 한번 만들면 값이 바뀌지 않는 객체를 짧게 선언하는 문법이다.

다음은 요청을 받아 처리하는 `PostController`다.

예제 1-3 `PostController.java`: 요청을 받고 글을 맡아 두는 컨트롤러

```
@Controller
@RequestMapping("/posts")
public class PostController {

    private final Map<Long, Post> posts = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong();

    @GetMapping
    public String list(Model model) {
```

```
        model.addAttribute("posts", posts.values());
        return "posts/list";
    }

    @GetMapping("/{id}")
    public String detail(@PathVariable Long id, Model model) {
        model.addAttribute("post", posts.get(id));
        return "posts/detail";
    }

    @GetMapping("/new")
    public String form() {
        return "posts/form";
    }

    @PostMapping
    public String create(@RequestParam String title,
            @RequestParam String content) {
        long id = sequence.incrementAndGet();
        posts.put(id, new Post(id, title, content, LocalDateTime.now()));
        return "redirect:/posts/" + id;
    }
}
```

데이터베이스 대신 메모리에 있는 `Map`에 글을 저장한다. 글을 쓰면 번호를 하나 늘려 저장하고, 목록 요청이 오면 저장된 글을 모두 화면에 넘긴다. 메서드가 돌려주는 `"posts/list"` 같은 문자열은 화면 파일의 이름이다. Spring은 `templates/posts/list.html`을 찾아 `Model`에 담긴 값을 채워 넣고 HTML로 바꿔 보낸다. 화면 파일은 이렇게 생겼다.

예제 1-4 `list.html`, `form.html`: 목록 화면과 글쓰기 화면

```
<!-- templates/posts/list.html -->
<h1>글 목록</h1>
<a href="/posts/new">새 글</a>
<ul>
  <li th:each="post : ${posts}">
    <a th:href="@{/posts/{id}(id=${post.id})}" th:text="${post.title}">제목</a>
    <span th:text="${#temporals.format(post.createdAt, 'yyyy-MM-dd')}">
      날짜</span>
  </li>
</ul>

<!-- templates/posts/form.html -->
<form method="post" action="/posts">
```

```
    <input name="title" placeholder="제목">
    <textarea name="content"></textarea>
    <button type="submit">저장</button>
  </form>
```

`th:`로 시작하는 속성이 화면 템플릿 엔진(Thymeleaf)의 문법이다. `th:each`는 목록을 돌며 줄을 만들고, `th:text`는 값을 글자로 넣는다. 두 파일 모두 `<head>` 같은 머리 부분은 줄였다. `detail.html`도 같은 방식으로 `${post.title}`, 작성 시각, `${post.content}`와 목록으로 돌아가는 링크를 보여 준다. 목록과 같은 모양이라 싣지 않았다. 저장소 v0의 `src/main/resources/templates/posts/detail.html`에 있다. 이 파일이 없으면 컨트롤러가 돌려준 `"posts/detail"`을 찾지 못해 상세 주소에서 500 오류가 난다.

앞 권에서 만든 조각을 찾아서

두원은 코드를 읽으며 앞 권 12장 끝에 적은 열 줄을 떠올렸다. 요청 하나가 소켓으로 들어와 응답으로 나가기까지 거치는 과정이다. 그 과정의 조각이 이 프로젝트에도 다 있었다. 다만 거의 보이지 않았다.

표 1-2 앞 권에서 직접 만든 것과 이 프로젝트에서 그 일을 하는 것

앞 권에서 직접 만든 것	이 프로젝트에서 그 일을 하는 것
따로 내려받아 실행한 Tomcat	Spring Boot 안에 든 내장 Tomcat
`WebInitializer`에서 직접 등록한 `DispatcherServlet`	Spring Boot가 시작할 때 자동으로 등록
JSP와 `ViewResolver`	Thymeleaf 템플릿(`templates/posts/*.html`)
`JdbcTemplate`과 MySQL 저장소	없다. 컨트롤러 안의 `Map` 하나

앞 권에서는 이 과정의 조각을 모두 직접 만들었고, 이번에는 한 줄도 쓰지 않았다. 마지막 줄이 마음에 걸렸다. 저장소가 없다. 앞 권 7장에서 `PostRepository`라는 약속을 세워 저장 방식을 컨트롤러에서 떼어 냈는데, AI는 글을 컨트롤러의 필드에 담았다. 이 선택이 어떤 문제를 부르는지는 4장과 10장에서 본다. 코드의 모양은 그림 1-3과 같다.

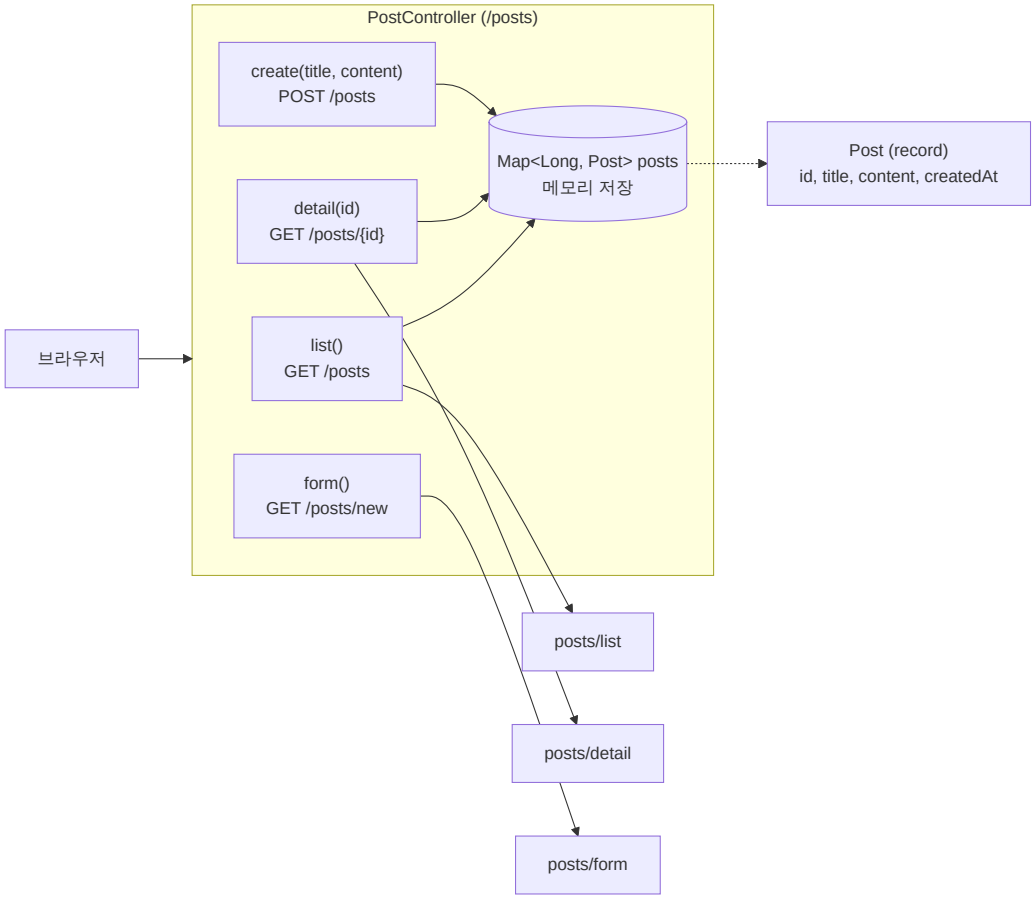


그림 1-3 v0의 코드 구조: 컨트롤러 하나가 요청 처리와 저장을 모두 맡는다

그렇다면 Spring은 `PostController`가 있는 줄 어떻게 알았을까? 앞 권 9장에서 두원은 클래스 목록을 훑어 컨트롤러를 찾아 만드는 장치를 직접 만들었다. 같은 일을 시작점의 annotation 하나가 한다.

`@SpringBootApplication`은 이 클래스가 있는 패키지(`blog`)와 그 아래 패키지를 훑어 `@Controller` 같은 표시가 붙은 클래스를 찾아 bean으로 등록한다. 거꾸로 말하면, 이 바깥에 놓인 클래스는 찾지 못한다. AI가 새 컨트롤러를 `com.example` 같은 다른 패키지에 만들면 서버는 아무 오류 없이 시작되고, 그 주소만 "찾을 수 없음"(404, 1.5절에서 다시 본다)이 된다.

코드는 멀쩡해 보이니 원인을 찾기 어렵다. AI가 만든 파일의 맨 윗줄 `package`가 시작점과 같은 패키지나 그 아래에 있는지 보는 습관을 들여 두자.

1.4 AI가 잘한 것

칭찬할 점부터 보자.

- **동작한다.** 요청한 세 가지 기능이 모두 실행된다.
- **이름이 좋다.** `Post`, `list`, `detail`, `create`는 무슨 일을 하는지 바로 알 수 있다.
- **관례를 따른다.** URL 구조(`/posts`, `/posts/{id}`, `/posts/new`)와 Spring의 annotation 사용이 흔히 쓰는 방식 그대로다.
- **동시성까지 신경 썼다.** 여러 요청이 동시에 와도 번호가 겹치지 않도록 `AtomicLong`과 `ConcurrentHashMap`을 썼다. 컨트롤러 객체 하나를 여러 스레드가 함께 쓴다는 것(앞 권 6장)을 AI는 알고 있었다.
- **새로고침에 대비했다.** 글을 저장한 뒤 화면을 바로 그리지 않고 상세 주소로 보낸다(`redirect:`). 그래서 저장 직후 새로고침해도 같은 글이 두 번 저장되지 않는다.

사람이 직접 작성했다면 이만큼 정돈된 코드를 10분 만에 내놓기는 어렵다. 두원은 앞 권에서 이만큼을 만드는 데 두 달 가까이 걸렸다. AI는 "흔히 이렇게 만든다"는 지식을 아주 많이 알고, 그 지식을 빠르게 코드로 옮긴다.

1.5 직접 시험해 보기: 어디가 위험한가

잘한 것은 화면에서 바로 보인다. 위험한 곳은 일부러 시험해 봐야 보인다. 두원은 마르코의 질문을 듣고 블로그를 실행한 채 하나씩 해 보았다. 1.2절에서 실행한 블로그로 그대로 따라 해 보자.

화면에서 해 볼 것

1. 글을 세 개 쓴다. 제목은 "첫째", "둘째", "셋째"로 한다. 목록에서 어느 글이 위에 있는가?
2. 제목을 비워 둔 채 본문만 쓰고 저장한다. 저장되는가? 목록에는 어떻게 보이는가?
3. 주소창에 `/posts/999`를 넣는다. 무엇이 보이는가?

4. 서버를 `Ctrl+C`로 종료하고 다시 실행한 뒤 목록을 연다. 쓴 글은 어디 갔는가?
5. 제목에 `<b>굵게</b>`를 넣어 저장한다. 목록에 굵은 글씨가 보이는가, 꺾쇠까지 글자로 보이는가?
6. 글을 저장한 직후 새로고침을 누른다. 같은 글이 두 번 생기는가?

두원의 화면에서는 "첫째"가 맨 위에 있었다. 최신 글이 위에 오리라던 기대와 반대다. 빈 제목의 글은 그대로 저장되어, 목록에 누를 글자가 없는 빈 줄로 보였다. `/posts/999`에서는 영어로 된 오류 페이지가 나타났다. 서버를 다시 켜자 글은 모두 사라졌다.

뒤의 둘은 안심되는 결과였다. `<b>굵게</b>`는 꺾쇠까지 글자 그대로 보였다. `th:text`는 값 속의 꺾쇠를 HTML 태그가 아닌 글자로 바꿔 넣는다. AI가 값을 HTML로 그대로 넣는 `th:utext`를 골랐다면, 누군가 제목에 스크립트를 넣는 순간 방문자의 브라우저에서 그 스크립트가 실행된다. 이 위험은 11장에서 마크다운을 붙일 때 다시 만난다. 새로고침해도 글은 하나뿐이었다. 1.4절에서 칭찬한 `redirect:` 덕분이다.

여섯 가지는 아무렇게나 고른 것이 아니다. 순서가 있을 법한 곳, 비어 있을 때, 없는 것을 찾을 때, 껐다 켤 때, 이상한 글자가 들어올 때, 같은 일을 두 번 할 때다. AI가 무엇을 만들었든 처음 시험해 볼 곳은 대개 이 근처에 있다. 3장에서는 이 목록이 AI가 사람에게 묻는 질문으로 발전한다.

요청과 응답을 직접 보기

화면은 결과만 보여 준다. 앞 권 3장에서 두원은 브라우저가 보내는 요청을 글자 하나하나까지 읽어 보았다. 저장 버튼을 누른 요청의 상태 코드와 헤더는 개발자 도구의 Network 탭에서도 보인다(Windows는 F12, macOS는 Cmd+Option+I로 열고 Network 탭을 고른다). 다만 화면은 정해진 요청만 보낸다. 칸을 비우거나 아예 빼는 식으로 요청을 일부러 바꿔 보내려면 터미널의 `curl`이 편하다. `-i`를 붙이면 응답의 상태 줄과 헤더까지 보인다. 세 요청의 결과를 먼저 모으면 그림 1-4와 같다. 하나씩 보내 보자.

터미널

```
  # 빈 제목으로 글쓰기 (화면의 저장 버튼이 보내는 요청과 같다)
$ curl -i -X POST http://localhost:8080/posts \
       --data-urlencode "title=" --data-urlencode "content=본문만 있는 글"

HTTP/1.1 302
Location: http://localhost:8080/posts/3
```

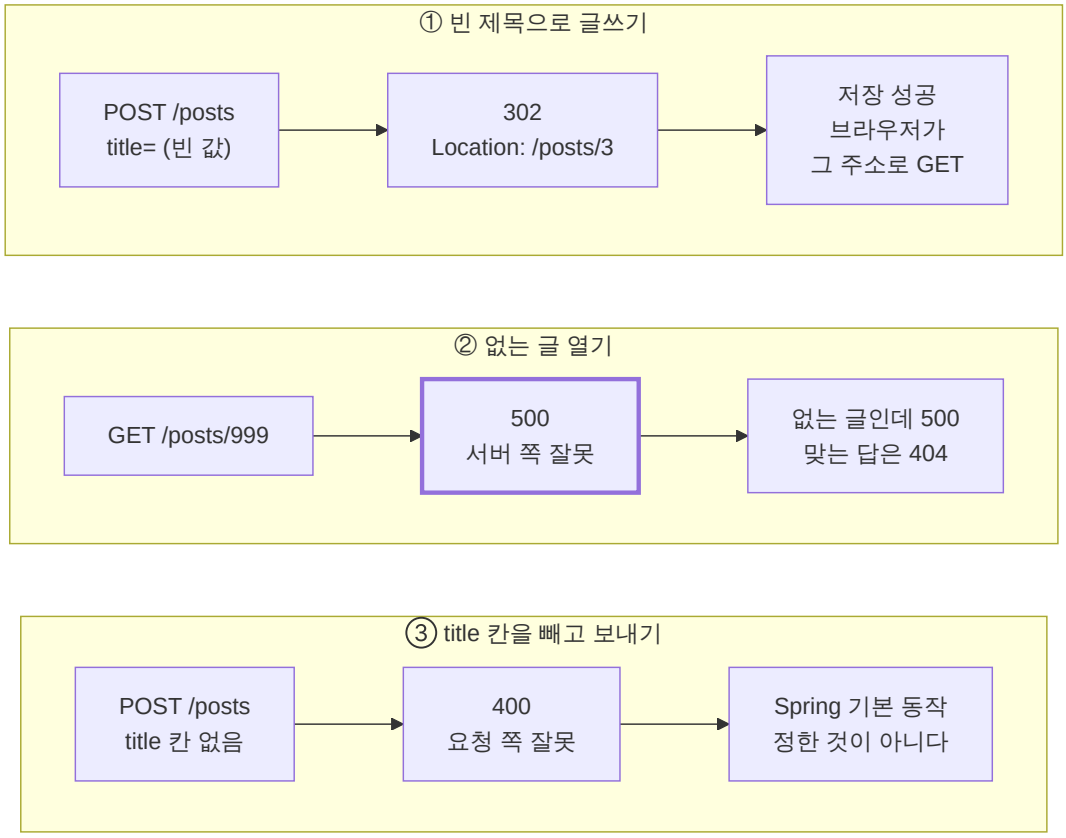


그림 1-4 curl로 본 요청과 응답: 저장은 302, 없는 글은 500, 칸이 빠진 요청은 400

저장은 성공했고, 이제 `Location`에 적힌 주소로 가라는 뜻이다. 1.4절에서 말한 "새로고침에 대비했다"가 바로 이 왕복이다. 그런데 두원의 눈길이 상태 코드에 멈췄다. 앞 권 5장에서 직접 만든 폼은 저장한 뒤 `303`을 돌려주었다. 앞 권 12장에서 Spring MVC로 옮길 때 `redirect:`가 조용히 `302`로 바꿔 놓은 것을 찾아낸 적도 있다. AI가 만든 코드도 같은 `302`였다. 브라우저에서는 똑같이 동작하니 지금 당장 문제는 아니다. 다만 AI는 어느 쪽을 고를지 묻지 않았다.

터미널

```
$ curl -i http://localhost:8080/posts/999
HTTP/1.1 500
Content-Type: text/html;charset=UTF-8
```

없는 글인데 `500`, 곧 서버 쪽 잘못이다. 서버 로그를 보면 `Property or field 'title' cannot be found on null`이 남아 있다. `posts.get(999)`가 `null`을 돌려주었고, 화면이 그 `null`에서 제목을 꺼내려다 오류가 발생한 것이다. 사람 눈에는 똑같은 오류 화면이지만, 앱이나 검색 엔진은 이 숫자를 보고 "서버가 고장 났다"고 판단한다. "그런 글은 없다"는 뜻의 `404`가 맞다.

터미널

```
  # title 칸을 아예 빼고 보내기
$ curl -i -X POST http://localhost:8080/posts \
       --data-urlencode "content=제목 칸이 없다"

HTTP/1.1 400
```

이번에는 `400`, 곧 요청 쪽 잘못이다. `@RequestParam`은 이름이 같은 칸이 요청에 없으면 요청을 거절한다. 그런데 칸이 있고 값이 비어 있으면 받아들인다. 빈 제목이 저장된 이유다. 이 요청이 거절된 것도 두원이 정해서가 아니라 Spring의 기본 동작 덕분이었다.

설정 파일도 본다

`pom.xml`은 길지 않다. Spring Initializr로 만들었다면 대략 이런 모양이다.

예제 1-5 `pom.xml`: Spring Initializr가 만든 빌드 설정

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>                <!-- 이 숫자를 확인한다 -->
</parent>

<groupId>blog</groupId>
<artifactId>blog</artifactId>
<!-- … (생략: version, name) -->

<properties>
    <java.version>21</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

이 책의 예제는 Spring Boot 3.5.16, 빌드 도구(Maven Wrapper) 3.9.11, JDK 21로 만들었다. 예제 저장소 v0 브랜치의 `pom.xml`이 이 모양이다. Spring Initializr가 이보다 새 버전을 골라 주었다면, 책과 같은 결과를 보려고 할 때는 숫자를 이 버전으로 고친다. `spring-boot-starter-parent` 아래의 버전을 Spring Initializr의 기본 버전과 비교한다. 1.2절에서 말했듯, AI가 뼈대를 만들었다면 이 숫자가 오래되었을 수 있다. 오래된 버전에는 이미 고쳐진 버그와 보안 문제가 남아 있다. 의존성 목록에 요청한 적 없는 라이브러리가 있는지도 본다. 모르는 이름이 있으면 "이 라이브러리는 왜 넣었어?"라고 묻는다.

이 확인이 왜 필요한지 두원은 곧 겪었다. 시험해 본 김에 빈 제목을 막아 달라고 AI에게 한 줄 맡겨 보았다. AI는 컨트롤러에 검사를 넣으면서 `pom.xml`에도 의존성 하나를 더했다.

```
<dependency>                                    <!-- AI가 더한 부분 -->
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
    <version>3.99.0</version>
</dependency>
```

`./mvnw spring-boot:run`은 서버가 시작되기도 전에 멈췄다.

```
[INFO] BUILD FAILURE
[ERROR] Failed to execute goal on project blog: Could not resolve dependencies …
[ERROR] dependency: org.apache.commons:commons-lang3:jar:3.99.0 (compile)
[ERROR]   Could not find artifact org.apache.commons:commons-lang3:jar:3.99.0
        in central (https://repo.maven.apache.org/maven2)
```

그런 버전은 없다. AI가 그럴듯한 숫자를 기억으로 지어 적었다. 오류를 붙여 주자 AI는 다른 숫자로 바꾸려 했다. 그런데 고칠 곳은 숫자가 아니었다. AI가 이 라이브러리에서 쓴 것은 빈 문자열 검사(`StringUtils.isBlank`) 하나였고, 자바에는 `String.isBlank()`가 이미 있다. 라이브러리 자체가 필요 없었다. 이번에는 빌드가 실패해서 눈에 띄었을 뿐이다. 버전이

맞았다면 필요 없는 라이브러리가 조용히 들어왔을 것이다. 두원은 이 시험을 첫 커밋으로 되돌렸다(되돌리는 법은 2장에서 본다).

그 뒤로 두원은 AI에게 일을 맡길 때 이 줄을 붙인다.

```
새 라이브러리가 필요하면 고치기 전에 먼저 물어봐.
이름과 버전, 왜 필요한지, 표준 라이브러리로는 안 되는지 적어 줘.
pom.xml에 이미 있는 버전 숫자는 바꾸지 마.
```

받은 결과는 diff에서 `pom.xml` 줄부터 본다. 이 파일의 한 줄은 코드 몇십 줄보다 영향이 크다. 프로젝트가 기대는 바탕이 바뀌기 때문이다. 그다음 실제로 어떤 라이브러리가 들어오는지 빌드 도구에게 직접 묻는다.

터미널

```
$ ./mvnw dependency:tree -Dscope=runtime
[INFO] blog:blog:jar:0.0.1-SNAPSHOT
[INFO] +- org.springframework.boot:spring-boot-starter-web:jar:3.5.16:compile
[INFO] |  +- org.springframework.boot:spring-boot-starter:jar:3.5.16:compile
…
[INFO] \- org.springframework.boot:spring-boot-starter-thymeleaf:jar:3.5.16:compile
```

`pom.xml`에는 실행에 쓰는 라이브러리를 두 개 적었는데 목록은 40줄 가까이 된다. 적은 라이브러리가 쓰는 라이브러리까지 따라 들어오기 때문이다. 이 목록은 19장에서 다시 쓴다. 지금은 AI가 다녀간 뒤 모르는 이름이 새로 생겼는지만 본다.

"새 라이브러리는 먼저 묻는다"는 두원이 AI에게 처음 정해 준 규칙이다. 3장에서 결정 기록이 생기면 그리로 옮기고, 15장에서는 AI context 문서의 "사람의 승인이 필요한 변경"에 "새 라이브러리 추가"로 들어간다. Gradle을 쓰든 다른 언어의 패키지 관리자를 쓰든 같다. AI가 고친 파일 중 의존성 설정 파일부터 읽고, 그 버전이 실제로 있는지와 그 라이브러리가 꼭 필요한지를 묻는다.

AI에게 물어보고, 그 답도 확인한다

시험해 본 뒤 두원은 AI에게 물었다.

방금 만든 블로그에서, 내가 정하지 않았는데 네가 정한 것을 모두 목록으로 알려 줘. 코드의 어느 줄인지도 함께.

AI는 금방 답했다.

1. 글은 메모리의 `ConcurrentHashMap`에 저장합니다. 서버를 다시 시작하면 사라집니다. (`PostController`의 `posts` 필드)
2. 글 번호는 1부터 하나씩 늘어납니다. (`AtomicLong`)
3. 목록은 글을 작성한 순서대로 보여 줍니다. (`list()`)
4. 작성 시각은 서버의 현재 시각으로 기록합니다. (`LocalDateTime.now()`)
5. 입력값 검증은 하지 않습니다.

쓸 만한 목록이다. 하지만 두원은 방금 시험해 본 결과와 나란히 놓고 두 군데에서 멈췄다.

셋째 줄은 사실과 다르다. `list()`에는 정렬하는 코드가 한 줄도 없다. `ConcurrentHashMap`은 꺼내는 순서를 약속하지 않는다. 지금 먼저 쓴 글이 위에 보이는 것은 글이 몇 개 없을 때 작은 번호가 저장 구조 안에 우연히 차례대로 놓인 결과일 뿐이다. 번호를 매기는 방식이나 저장 구조가 바뀌면 순서도 달라진다. AI는 "지금 그렇게 보인다"를 "그렇게 만들었다"로 적었다.

그리고 빠진 줄이 있다. 없는 글에 `500`을 돌려준다는 것은 목록 어디에도 없다.

두원은 같은 대화에서 다시 물었다. "3번은 `list()`의 어느 줄에서 작성 순서로 정렬해?" 이번에는 답이 달랐다.

확인해 보니 `list()`에는 정렬 코드가 없습니다. `ConcurrentHashMap`은 순서를 보장하지 않으므로, 지금 작성 순서로 보이는 것은 우연입니다. 3번을 "목록 순서는 정해져 있지 않습니다"로 고칩니다. 또 없는 글 번호로 들어오면 화면에서 오류가 나 500 응답이 갑니다. 이것도 목록에 더합니다.

물으면 고친다. 빠진 줄도 찾아 더했다. 하지만 묻지 않았다면 틀린 설명이 그대로 남았을 것이다. AI가 쓴 코드에 대한 AI의 설명도 하나의 주장이다. 주장은 코드와 실행 결과로 확인한다. 이 장에서 두원이 처음 익힌 습관이다.

1.6 아무도 정하지 않은 것들

이제 마르코의 질문으로 돌아가 보자. 목록은 어떤 순서로 보일까?

1.5절에서 본 대로, 코드는 `posts.values()`를 그대로 넘긴다. 글이 몇 개 없을 때는 우연히 쓴 순서대로, 곧 오래된 글이 위에 보인다. 그 순서조차 저장 구조가 우연히 만든 것이다. 최신 글이 위에 오리라는 두원의 기대는 코드 어디에도 없다.

AI가 실수한 걸까? 그렇지 않다. 두원은 순서를 말하지 않았다. AI는 말하지 않은 것을 자기가 아는 가장 무난한 방식으로 채웠을 뿐이다.

같은 방법으로 시험해 본 결과를 모으면, 정하지 않은 것이 더 보인다.

표 1-3 아무도 정하지 않은 것들

질문	지금 코드의 동작	누가 정했나
목록은 어떤 순서로 보이나?	순서 보장 없음	아무도
제목 없이 글을 쓸 수 있나?	빈 제목도 저장됨	아무도
없는 글 번호로 들어가면?	500 오류 페이지	아무도
서버를 다시 켜면 글은?	모두 사라짐	아무도
글을 고치거나 지울 수 있나?	기능 없음	두원이 말하지 않음

표의 마지막 열은 대부분 "아무도"다. AI가 대신 정한 것도 아니다. 아무도 정하지 않은 빈칸을 코드가 우연한 동작으로 채우고 있을 뿐이다.

두원은 이 말이 정말인지 확인해 보고 싶었다. 새 대화를 열고 1.1절의 한 줄을 그대로 다시 넣었다. 두 번째 블로그도 잘 돌아갔다. 그런데 같은 질문으로 칸을 다시 채워 보니 답이 달랐다.

표 1-4 같은 한 줄 요청으로 만든 두 블로그

질문	첫 번째 블로그	두 번째 블로그
목록은 어떤 순서로 보이나?	순서 보장 없음	글 번호를 거꾸로 정렬해 보여 준다
제목 없이 글을 쓸 수 있나?	빈 제목도 저장된다	화면의 입력란에서만 막는다. 서버는 그대로 받는다
없는 글 번호로 들어가면?	500 오류 페이지	말없이 목록으로 돌려보낸다
글 번호는 어떻게 생겼나?	1부터 하나씩 는다	긴 무작위 문자열이다

두 번째 블로그의 컨트롤러에서 해당하는 줄만 뽑으면 이렇다.

예제 1-6 PostController.java: 두 번째 블로그에서 달라진 줄

```
// 글 번호: 긴 무작위 문자열
String id = UUID.randomUUID().toString();

// 목록: 번호 순으로 정렬해 뒤집는다
model.addAttribute("posts", posts.values().stream()
        .sorted(Comparator.comparing(Post::id).reversed())
        .toList());

// 상세: 없으면 조용히 목록으로
Post post = posts.get(id);
if (post == null) {
    return "redirect:/posts";
}
```

번호를 거꾸로 정렬하니 최신 글이 위에 오는 것처럼 보인다. 그런데 비교하는 번호는 무작위 문자열이다. 두원이 확인한 날에는 우연히 최신 글이 위에 보였을 뿐, 문자열의 크기는 글을 쓴 순서와 아무 관계가 없다. 그리고 화면의 `required`는 브라우저가 지키는 약속일 뿐이라, 1.5절의 `curl`처럼 화면을 거치지 않는 요청은 그대로 통과한다.

글쓰기, 목록, 상세 보기는 두 번 다 있었다. 두원이 말한 것이다. 말하지 않은 칸은 실행할 때마다 다르게 채워졌다. **같은 요청에서 다른 결과가 나온 것이다.** AI는 같은 질문에도 늘 같은 답을 내지는 않는다. 답을 만들 때마다 여러 그럴듯한 선택지 가운데 하나를 고르기 때문이다. 요청에 적은 것은 고를 수 있는 범위를 좁히지만, 적지 않은 부분에서는 그때그때 다른 선택지가 뽑힌다.

그러니 "아무도 정하지 않았다"는 말은, 정확히는 "AI가 매번 다르게 정한다"는 뜻이다. 오늘 마음에 든 동작도 내일 같은 요청을 다시 실행하면, 혹은 다른 기능을 고쳐 달라고 하는 사이에 바뀔 수 있다. 다음에도 그대로 남는 것은 정해서 말한 것뿐이다.

좋은 코드와 원하는 코드는 다르다. 이 블로그의 코드는 깔끔하지만, 두원이 원하는 블로그인지는 아직 아무도 모른다. 두원 자신도 모른다.

1.7 개발자의 일은 어디로 갔나

두원은 처음에 "이제 개발은 AI가 하는구나"라고 생각했다. 절반은 맞다. 코드를 타이핑하는 일은 확실히 AI가 가져갔다. 하지만 표 1-3의 빈칸을 채우는 일은 여전히 남아 있다. 목록은 최신순이어야 하는가, 인기순이어야 하는가? 빈 제목은 막아야 하는가? 없는 글에는 어떤 화면을 보여 줘야 하는가?

이 질문에는 정답이 없다. 두원이 어떤 블로그를 원하느냐에 따라 답이 달라지니, AI가 대신 답할 수 없다. 1.6절의 두 블로그 중 어느 쪽 순서가 맞는지도 AI가 아니라 두원이 정할 일이다. 이 질문에 답하고, 그 답을 AI에게 정확히 전하고, 결과가 답대로 만들어졌는지 확인하는 것. 이것이 AI 시대에 개발자에게 남는 일이다.

같은 기능, 두 가지 요청

그냥 요청: "Spring Boot로 간단한 블로그 만들어 줘. 글쓰기, 목록, 상세 보기가 있으면 돼."

```
// list(): 순서는 우연에 맡긴다
model.addAttribute("posts", posts.values());

// detail(): 없는 글이면 null을 넘긴다
model.addAttribute("post", posts.get(id));
```

정해서 요청: "같은 블로그를 만들어 줘. 목록은 작성 시각 기준 최신 글이 위에 온다. 빈 제목은 저장하지 않는다. 없는 글 번호로 들어오면 404 페이지를 보여 준다."

```
// list(): 최신 글이 위로
model.addAttribute("posts", posts.values().stream()
        .sorted(Comparator.comparing(Post::createdAt).reversed())
        .toList());

// detail(): 없는 글이면 404
Post post = Optional.ofNullable(posts.get(id))
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
```

무엇이 달라졌나: 정한 것은 코드에 한 줄씩 흔적을 남긴다. 그 줄만 보면 무엇을 정했는지 알고 확인할 수 있다. 정하지 않은 것은 흔적이 없어서, 그냥 요청의 결과는 아무리 읽어도 무엇이 빠졌는지 보이지 않는다.

정해서 요청한 결과도 1.5절의 방법으로 똑같이 확인한다. 글 세 개를 쓰고 순서를 보고, `curl -i`로 `/posts/999`를 불러 `404`가 오는지 본다. 빈 제목은 화면이 아니라 `curl`로 보내 본다. 요청에 적은 문장마다 확인할 방법이 하나씩 있다는 것도 눈여겨보자. 이 짝은 7장에서 테스트가 된다.

그렇다고 처음부터 이렇게 요청했어야 한다는 말은 아니다. 두원은 만들어 보기 전까지 순서를 정해야 한다는 사실조차 몰랐다. 빈칸은 만들어 봐야 보인다. 계획 없이 시작한 것은 잘못이 아니라 자연스러운 출발점이다.

돌아보면 두원은 이 장에서 AI와 일하는 과정을 한 번 끝까지 거쳤다. 맡기고, 실행하고, 시험해 보고, AI의 설명을 실행 결과와 맞춰 보고, 커밋했다. 다음 장부터 이 과정이 조금씩 꼼꼼해진다. 맡기기 전에 무엇을 정할지, 맡긴 뒤에 무엇을 읽을지가 하나씩 더해진다.

개발 상식 Spring Boot가 더한 것, Maven Wrapper

Spring Boot가 더한 것. 앞 권 12장에서는 `DispatcherServlet`을 직접 등록하고, `ViewResolver`와 `DataSource`를 bean으로 직접 적고, WAR로 묶어 Tomcat에 올렸다. Spring Boot는 이 일을 대신한다. classpath에 Spring Web이 있으면 내장 Tomcat과 `DispatcherServlet`을, Thymeleaf가 있으면 템플릿을 찾는 `ViewResolver`를 알아서 만든다. 이것을 자동 구성(auto-configuration)이라고 한다. 편한 만큼 무엇이 만들어졌는지 보이지 않는다. 궁금하면

`application.yml`에 `debug: true`를 적고 실행해 보자. 어떤 자동 구성이 쓰였고 어떤 것이 빠졌는지 로그에 나온다.

프로젝트 구조. AI가 만든 프로젝트도 대개 다음 모양이다. 구조를 알면 AI가 만든 파일이 어디에 왜 있는지 읽을 수 있다.

```
src/main/java/blog/BlogApplication.java   애플리케이션 시작점
src/main/java/blog/PostController.java    요청 처리
src/main/resources/templates/             화면 템플릿
src/main/resources/application.yml        설정
src/test/java/                            테스트 코드
pom.xml                                   의존성과 빌드 설정
mvnw, mvnw.cmd, .mvn/wrapper/             빌드 도구 wrapper (커밋한다)
target/                                   빌드 결과물 (커밋하지 않는다)
```

Maven Wrapper. 앞 권 1장에서는 Maven을 컴퓨터에 설치하고 `mvn`으로 실행했다. 이 책은 저장소에 든 wrapper(`mvnw`)로 실행한다. `mvnw`는 `.mvn/wrapper/maven-wrapper.properties`에 적힌 Maven 버전(이 책은 3.9.11)을 처음 한 번 내려받아 쓴다. 그래서 따로 설치하지 않아도 내 컴퓨터와 CI 서버가 같은 버전으로 빌드한다. wrapper 파일은 커밋한다. 하는 일은 앞 권과 같다. 의존성을 내려받고, 컴파일하고, 테스트를 실행하고, JAR로 묶는다. `<scope>`를 적지 않은 의존성은 코드가 쓰는 것, `test`는 테스트만 쓰는 것, `runtime`은 실행할 때만 필요한 것이다. `<parent>`의 `spring-boot-starter-parent`가 Spring Boot와 맞는 라이브러리 버전을 정해 두므로, 의존성마다 버전을 적지 않아도 된다.

이 장에서 배운 것

항목	핵심
한 줄 요청의 결과	AI는 한 줄 요청만으로도 동작하고 읽기 좋은 코드를 만든다. 앞 권에서 직접 만든 조각(Tomcat, `DispatcherServlet`, 뷰)은 Spring Boot 안에 숨어 있고, 저장소는 아예 없다.
실행하고 시험하기	AI가 만든 프로젝트는 직접 실행하고, 화면과 `curl -i`로 시험하고, 설정 파일의 버전까지 확인한 뒤 첫 커밋으로 남긴다. 순서, 빈 값, 없는 것, 재시작, 이상한 글자, 두 번 하기가 처음 시험해 볼 곳이다.

항목	핵심
의존성 설정	AI가 고친 파일 중 `pom.xml` 같은 의존성 설정부터 읽는다. 새 라이브러리는 고치기 전에 먼저 묻게 하고, 버전이 실제로 있는지와 꼭 필요한지를 확인한다.
AI의 설명	`BUILD SUCCESS`는 확인한 것이 맞았다는 뜻이지, 무언가를 확인했다는 뜻이 아니다. AI가 자기 코드를 설명한 말도 하나의 주장이므로 코드와 실행 결과로 확인한다.
정하지 않은 것	요청에 없던 결정(순서, 입력 규칙, 오류 처리, 저장 방식)은 아무도 내리지 않은 채 남고, 같은 요청을 다시 실행하면 다르게 채워진다. 다음에도 그대로 남는 것은 정해서 말한 것뿐이다.

이 장의 코드는 예제 저장소 `blog-with-ai-source`의 `v0` 브랜치에 있다(부록 E).

연습 문제

1. 1.2절을 따라 프로젝트를 만들고, 1.1절의 프롬프트로 블로그를 만든 뒤 첫 커밋까지 해 보자. 새 대화에서 같은 프롬프트를 한 번 더 실행해, 두 결과가 어디서 갈리는지도 보자.
2. 1.5절의 화면 확인 여섯 가지와 `curl -i` 요청 세 가지를 자신의 블로그에 해 보자. 상태 코드는 각각 무엇인가?
3. AI에게 "내가 정하지 않았는데 네가 정한 것"을 물어보고, 그 답을 코드와 실행 결과로 한 줄씩 확인해 보자. 사실과 다른 줄이나 빠진 줄이 있는가?
4. 표 1-3을 자신의 코드로 다시 채워 보자. "누가 정했나" 열이 "아무도"인 칸은 몇 개인가? 그중 하나를 골라, 자신이 원하는 동작을 한 문장으로 적어 보자. 3장에서 결정 기록을 쓸 때 이 문장을 다시 꺼낸다.
5. AI에게 작은 기능 하나를 맡기기 전과 뒤에 아래 명령을 실행해 출력을 비교해 보자. 새로 생긴 이름이 있다면 왜 필요한지 AI에게 묻고, 표준 라이브러리로 되는지 확인하자.

터미널

```
$ ./mvnw dependency:tree -Dscope=runtime
```

마르코의 질문

상황: AI로 만든 시제품을 본 팀장이 묻는다.

"하루 만에 됐네요. 그럼 다음 주에 바로 내보낼 수 있죠?"

짚어야 할 것

- 동작하는 것과 내보낼 수 있는 것은 다르다. 1.6절의 "아무도 정하지 않은 것" 표를 꺼내 보여 주는 것이 가장 빠른 설명이다.
- 표의 줄마다 누가 정해야 하는지 나눈다. 개발자가 정할 것(재시작하면 글이 사라지는 문제), 기획이나 운영이 정할 것(빈 제목을 허용할지)이 섞여 있다.
- 남은 일정은 코드를 쓰는 시간이 아니라 정하고 확인하는 시간으로 잡는다.

빠지기 쉬운 답: "네, AI가 다 만들어 줬어요." 또는 "코드 품질이 아직이라서요." 문제는 코드의 품질보다 정하지 않은 것이 남아 있다는 데 있다.

. . .

두원 정리해 보니까 표의 칸이 거의 다 "아무도"야. 내가 직접 만들 때는 이런 칸이 없었는데.

마르코 없었던 게 아니라, 네가 코드를 쓰면서 하나씩 정했던 거지. `ORDER BY`를 쓸 때 순서를 정했고, 404를 돌려줄 때 없는 글을 어떻게 할지 정했고.

두원 아, 그러네. 코드를 안 쓰니까 정할 기회도 같이 사라진 거구나.

마르코 응. 그 기회를 따로 만들어야 해. 다음 주에 기능 하나만 더 넣어 봐. 그때 이 빈칸들이 어떻게 다시 문제가 되는지 보일 거야.

2
장

“임시저장도 넣어 줘” 했더니 미완성 글이 목록에 나왔다

2.1 두 가지 요청

2.2 AI가 바꾼 코드

2.3 무엇이 잘못되었나

2.4 AI는 틀리지 않았다

2.5 그냥 “고쳐 줘”라고 하면?

2.6 AI가 바꾼 것을 읽고 나누기

2.7 되돌리고, 브랜치에서 시도하기

2.8 개발자의 일이 바뀐다

이 장에서 배울 것

- AI가 만든 결과가 어긋났을 때, 문제가 AI가 아니라 요청에 있었는지 가려낼 수 있다.
- AI가 바꾼 코드를 diff로 읽고, 성격이 다른 변경을 나눠 커밋할 수 있다.
- 커밋과 브랜치로 AI의 시도를 되돌리고, AI에게 넘기면 안 되는 정보를 판단할 수 있다.
- AI와 개발할 때 개발자의 일을 정하기, 말하기, 확인하기로 나눠 설명할 수 있다.

이 장의 실습

- **저장소:** `blog-with-ai-source`
- **브랜치:** `v0`에서 시작해 `v1`과 비교
- **만들 것:** 블로그 v1(임시저장, 제목으로 된 주소), 나눠 커밋한 기록, 시도용 브랜치

일요일 밤, 두원이 마르코에게 메시지를 보냈다.

> **두원** 큰일 났어. 민찬이가 내 블로그에 들어왔다가 "이 글 왜 쓰다 말았어?"라고 물어봤어. 임시저장해 둔 글이 목록에 그대로 나와 있었대.
>
> **마르코** 어, 그거 AI한테 만들어 달라고 했다던 그 기능?
>
> **두원** 응. 분명히 만들어 줬어. 저장도 잘되고.
>
> **마르코** 그럼 AI는 시킨 일을 했네. 너는 뭘 시켰는데?
>
> **두원** "임시저장도 넣어 줘."
>
> **마르코** 그 문장 어디에 "임시저장한 글은 다른 사람에게 보이면 안 된다"가 들어 있어?
>
> **두원** …없네.

· · ·

2.1 두 가지 요청

1장의 블로그를 며칠 써 본 두원은 불편한 점 두 가지를 발견했다.

첫째, 글을 한 번에 다 쓰기 어려웠다. 쓰다가 저장해 두고 나중에 이어 쓰고 싶었다. 둘째, 글 주소가 `/posts/1`, `/posts/2`처럼 숫자라서 친구에게 링크를 보내면 무슨 글인지 알 수 없었다.

주소에 제목이 들어가면 좋겠다고 생각했다.

요청하기 전에 두원은 `git status`부터 실행했다. `nothing to commit, working tree clean`. 1장의 첫 커밋 뒤로 바뀐 것이 없다는 뜻이다. 이제 AI가 무엇을 바꾸든, 바뀐 것은 모두 이번 요청 때문이다. 그리고 AI에게 이렇게 요청했다.

임시저장도 넣어 줘. 그리고 글 주소를 숫자 대신 제목으로 바꿔 줘.

이번에도 AI는 빠르게 일을 마쳤다. 글쓰기 화면에 "임시저장" 버튼이 생겼고, 목록의 글 주소가 `/posts/첫-번째-글`처럼 바뀌었다. 이렇게 제목을 주소에 쓸 수 있게 바꾼 문자열을 **슬러그(slug)**라고 부른다. AI의 보고는 이랬다.

두 가지를 추가했습니다.

1. 임시저장: 글쓰기 화면에 "임시저장" 버튼을 추가했습니다. `Post`에 `draft` 필드를 두어 임시저장 글을 구분합니다.
2. 제목 주소: 제목으로 슬러그를 만드는 `SlugUtils`를 추가했습니다. 글 상세 주소가 `/posts/{slug}`로 바뀝니다.

기존 기능은 그대로 동작합니다.

마지막 줄이 눈에 띈다. "기존 기능은 그대로 동작합니다." 목록도 그대로라는 뜻이다. 이 말이 무슨 뜻이었는지는 곧 드러난다.

2.2 AI가 바꾼 코드

`Post`에는 필드가 두 개 늘었다.

예제 2-1 `Post.java`: 슬러그와 임시저장 필드가 늘어난 글

```
public record Post(Long id, String title, String slug,
                   String content, boolean draft,
                   LocalDateTime createdAt) {
}
```

`slug`는 주소에 쓸 문자열이고, `draft`는 임시저장 글인지를 나타낸다. 슬러그는 새로 생긴 `SlugUtils` 클래스가 만든다.

예제 2-2 SlugUtils.java: 제목으로 슬러그 만들기

```
public final class SlugUtils {

    private SlugUtils() {
    }

    public static String toSlug(String title) {
        return title.trim().toLowerCase().replaceAll("\\s+", "-");
    }
}
```

앞뒤 공백을 지우고, 소문자로 바꾸고, 공백을 하이픈(`-`)으로 바꾼다. 컨트롤러의 글쓰기와 상세 보기도 바뀌었다.

예제 2-3 PostController.java: 슬러그로 쓰고 읽는 글쓰기와 상세 보기

```
@PostMapping
public String create(@RequestParam String title,
                     @RequestParam String content,
                     @RequestParam(defaultValue = "false") boolean draft) {
    long id = sequence.incrementAndGet();
    String slug = SlugUtils.toSlug(title);
    posts.put(id, new Post(id, title, slug, content, draft,
            LocalDateTime.now()));
    return "redirect:/posts/" + slug;
}

@GetMapping("/{slug}")
public String detail(@PathVariable String slug, Model model) {
    Post post = posts.values().stream()
            .filter(p -> p.slug().equals(slug))
            .findFirst()
            .orElse(null);
    model.addAttribute("post", post);
    return "posts/detail";
}
```

글쓰기 화면에는 버튼이 하나 늘었다. 같은 폼에 버튼이 둘이고, 누른 버튼의 `name`과 `value`만 요청에 실려 간다.

```
<button type="submit">발행</button>
<button type="submit" name="draft" value="true">임시저장</button>
```

"발행"을 누르면 `draft` 칸이 요청에 없으니 `defaultValue = "false"`가 쓰이고, "임시저장"을 누르면 `draft=true`가 간다. 목록을 보여 주는 `list` 메서드는 바뀌지 않았다. 1장과 똑같이 저장된 글을 모두 화면에 넘긴다.

diff로 읽기

두원은 화면에서 버튼 두 개를 눌러 보고, 커밋하기 전에 무엇이 바뀌었는지 Git으로 확인했다. 먼저 바뀐 규모부터 본다.

터미널

```
$ git diff --stat
 src/main/java/blog/Post.java                 |  5 +++--
 src/main/java/blog/PostController.java       | 20 ++++++++++++++-------
 src/main/java/blog/SlugUtils.java            |  8 ++++++++
 src/main/resources/templates/posts/form.html |  3 ++-
 src/main/resources/templates/posts/list.html |  3 ++-
 5 files changed, 28 insertions(+), 11 deletions(-)
```

파일 다섯 개, 더한 줄 28개, 지운 줄 11개다. 이 정도면 다 읽을 수 있다. 이제 파일마다 내용을 연다.

```
$ git diff src/main/java/blog/Post.java
diff --git a/src/main/java/blog/Post.java b/src/main/java/blog/Post.java
index 3c1a9e2..8f04b7d 100644
--- a/src/main/java/blog/Post.java
+++ b/src/main/java/blog/Post.java
@@ -2,6 +2,7 @@ package blog;

 import java.time.LocalDateTime;

-public record Post(Long id, String title, String content,
-        LocalDateTime createdAt) {
```

```
+public record Post(Long id, String title, String slug,
+                    String content, boolean draft,
+                    LocalDateTime createdAt) {
 }
```

읽는 법은 단순하다. `---`는 바뀌기 전 파일, `+++`는 바뀐 뒤 파일이다. `@@ -2,6 +2,7 @@`는 "바뀌기 전 파일의 2번째 줄부터 6줄이, 바뀐 뒤에는 2번째 줄부터 7줄이 되었다"는 뜻이다. 필드가 늘어 선언이 한 줄 길어졌다. 그 아래에서 `-`로 시작하는 줄은 지워졌고, `+`로 시작하는 줄은 더해졌으며, 공백으로 시작하는 줄은 위치를 알려 주려고 함께 보여 주는 바뀌지 않은 줄이다. 한 줄을 고치면 "옛 줄을 지우고 새 줄을 더한" 모양으로 나온다.

두원은 다섯 파일을 차례로 읽었다. `list.html`의 두 줄은 링크 주소를 `id`에서 `slug`로 바꾼 것이었다. 요청과 맞다. 모두 확인하고 커밋했다.

터미널

```
$ git add .
$ git commit -m "임시저장과 제목 주소 추가"
```

그런데 두원은 이 diff에서 중요한 것 하나를 놓쳤다. **diff에는 바뀐 것만 보이고, 바뀌었어야 하는데 바뀌지 않은 것은 보이지 않는다.** `list.html`은 바뀌었지만 `list()` 메서드는 한 줄도 바뀌지 않았다. 그 사실은 diff 어디에도 나오지 않는다. AI 보고의 "기존 기능은 그대로 동작합니다"라는 말이 바로 그 빈자리를 가리키고 있었다.

2.3 무엇이 잘못되었나

코드는 이번에도 깔끔하다. 그런데 블로그를 며칠 더 쓰자 문제가 하나씩 드러났다.

임시저장 글이 목록에 보인다. `list` 메서드는 `draft` 값을 보지 않는다. 쓰다 만 글도, 다 쓴 글도 똑같이 목록에 오른다. 민찬이 본 것이 이것이다.

같은 제목의 글은 하나만 보인다. 두원은 매주 "주간 회고"라는 제목으로 글을 쓰기로 했다. 두 번째 주에 같은 제목으로 글을 쓰자, 두 글 모두 슬러그가 `주간-회고`가 되었다. 상세 보기는 슬러그가 같은 글 중 아무거나 첫 번째 것을 보여 준다. 1장에서 본 것처럼 `ConcurrentHashMap`은 순서를 보장하지 않으니, 어느 쪽이 보일지도 알 수 없다. 나머지 한 글은 주소로 찾아갈 방법이 없다.

특수 문자가 들어간 제목은 열리지 않는다. "C# 입문?"이라는 글을 쓰면 슬러그는 `c#-입문?`이 된다. 그런데 주소에서 `#`은 페이지 안의 위치를, `?`는 검색 조건의 시작을 뜻한다. 브라우저는 `/posts/c`까지만 서버에 요청하고, 서버는 그런 글이 없다고 답한다.

직접 재현해 보기

세 가지 모두 브라우저로 볼 수 있다. 글쓰기 화면에서 임시저장 글을 하나 쓰고 `http://localhost:8080/posts`를 열면 목록에 그 글이 보인다(그림 2-1). 같은 제목으로 두 번 쓰고 그 글을 열면 한 글만 열린다. 같은 증상을 직접 다시 만들 수 있어야, 고친 뒤에 정말 고쳐졌는지도 확인할 수 있다. 고칠 때마다 화면을 다시 누르기는 번거로우니, 같은 확인을 1장에서 쓴 `curl` 명령으로 적어 둔다. 명령은 몇 번이고 똑같이 다시 실행할 수 있다.



그림 2-1 미완성 글이 목록에 노출된 화면

터미널

```
  # 1. 임시저장 글을 하나 쓰고 목록에 나오는지 본다
$ curl -s -X POST http://localhost:8080/posts \
       --data-urlencode "title=쓰다 만 글" \
       --data-urlencode "content=..." -d "draft=true" -o /dev/null

  # 한 줄이 나오면 노출된 것
$ curl -s http://localhost:8080/posts ¦ grep "쓰다 만 글"

  # 2. 같은 제목으로 두 번 쓴다
$ for i in 1 2; do
    curl -s -X POST http://localhost:8080/posts \
         --data-urlencode "title=주간 회고" \
         --data-urlencode "content=${i}주차" -o /dev/null
  done
$ curl -s http://localhost:8080/posts/%EC%A3%BC%EA%B0%84-%ED%9A%8C%EA%B3%A0 \
       ¦ grep "주차"
  # 1주차와 2주차 중 하나만 나온다
```

마지막 `curl`의 낯선 주소가 `/posts/주간-회고`다. 주소에는 영문, 숫자와 몇몇 기호만 쓸 수 있어서, 한글은 UTF-8 바이트로 바꾼 뒤 바이트마다 `%`와 두 자리 16진수로 적는다. `주간`은 `%EC%A3%BC%EA%B0%84`가 된다. 이것을 퍼센트 인코딩이라고 부른다. 브라우저 주소창은 보기 좋게 한글로 보여 줄 뿐이다.

`#`과 `?`처럼 주소에서 특별한 뜻을 가진 문자도 글자 그대로 쓰려면 `%23`, `%3F`로 바꿔야 한다. `/posts/c%23-%EC%9E%85%EB%AC%B8%3F`로 요청하면 "C# 입문?" 글이 열린다. 목록 화면의 링크는 템플릿이 알아서 이렇게 바꿔 준다. 문제는 바꿔 주지 않는 곳이다. 글을 저장한 뒤 상세로 보내는 `"redirect:/posts/" + slug`는 슬러그를 그대로 이어 붙인다. 그래서 저장 버튼을 누르자마자 `/posts/c`로 가 버린다.

주소창의 주소를 직접 고쳐 친구에게 보낼 때도 같은 일이 생긴다. 한글도 다르지 않다. 인코딩하지 않은 한글 주소는 내장 Tomcat이 이동할 주소를 담는 응답 헤더(`Location`)에 싣지 않는다. 그래서 "주간 회고"를 저장하면 이동할 곳이 빠진 응답이 와서 브라우저가 빈 화면에 멈춘다. 목록에서 링크를 누르면 잘 열리니 놓치기 쉽다. 고치는 방법은 둘이다. 주소를 만드는 모든 곳에서 빠짐없이 인코딩하거나, 처음부터 주소에 문제가 없는 문자만 슬러그에 남기거나. 어느 쪽을 고를지는 3장에서 두원이 정한다.

2.4 AI는 틀리지 않았다

두원은 억울했다. "AI가 대충 만든 거 아니야?"

정말 그럴까? 두원의 요청을 다시 읽어 보자.

임시저장도 넣어 줘. 그리고 글 주소를 숫자 대신 제목으로 바꿔 줘.

이 문장에 들어 있는 것과 들어 있지 않은 것을 나눠 보면 이렇다.

표 2-1 두원이 말한 것과 말하지 않은 것

두원이 말한 것	두원이 원했지만 말하지 않은 것
임시저장 기능을 넣는다	임시저장 글은 나에게만 보인다
주소에 제목을 쓴다	같은 제목이어도 글마다 주소가 달라야 한다
	어떤 제목이든 주소로 열 수 있어야 한다

AI는 왼쪽 열을 정확히 만들었다. 오른쪽 열은 두원의 머릿속에만 있었다. 아니, 사실은 두원의 머릿속에도 없었다. 민찬이 묻기 전까지 두원은 "임시저장 글을 누가 볼 수 있는가"를 생각해 본 적이 없다. 원한 것, 말한 것, 만든 것을 나란히 놓으면 그림 2-2와 같다.

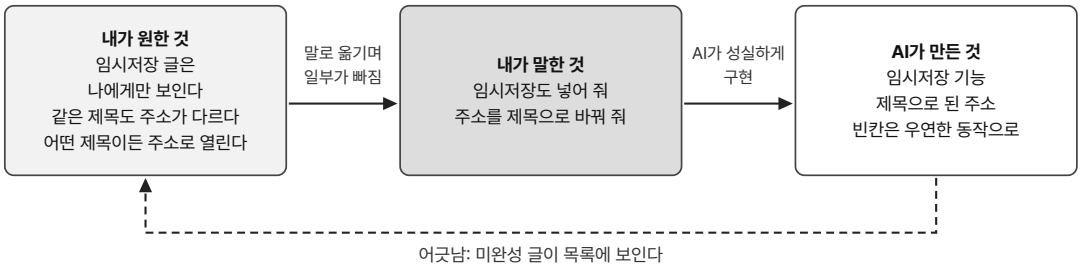


그림 2-2 내가 원한 것, 내가 말한 것, AI가 만든 것

이 그림은 이 책 전체에서 반복될 장면이다. 원한 것은 말로 옮기는 순간 일부가 빠지고, AI는 말한 것을 성실하게 만들며, 빠진 부분은 우연한 동작으로 채워진다. AI가 똑똑해질수록 "말한

것”은 더 정확하게 만들겠지만, “원한 것”과 “말한 것” 사이의 틈은 AI가 메울 수 없다. 그 틈은 내 쪽에 있기 때문이다.

2.5 그냥 “고쳐 줘”라고 하면?

두원은 AI에게 다시 요청했다.

임시저장한 글이 목록에 나와. 고쳐 줘.

AI의 답은 짧았다.

목록에서 임시저장 글이 보이지 않도록 `list()`에 필터를 추가했습니다. 이제 목록에는 발행된 글만 표시됩니다.

목록 메서드에 임시저장 글을 걸러 내는 `filter` 한 줄이 들어갔다. 2.3절의 첫 번째 `curl`을 다시 실행하자 아무것도 나오지 않았다. 문제가 해결된 것처럼 보인다. 하지만 상세 보기는 그대로다. 임시저장 글의 주소를 아는 사람은 여전히 그 글을 열 수 있다. 슬러그가 제목에서 나오니, 제목만 짐작하면 주소도 짐작할 수 있다.

이번에는 같은 제목 문제를 말했다.

같은 제목으로 글을 쓰면 하나만 보여. 고쳐 줘.

AI는 슬러그가 겹치면 뒤에 숫자를 붙이도록 고쳤다. 컨트롤러에 반복문이, `SlugUtils`에 숫자를 붙이는 메서드가 생겼다.

예제 2-4 `PostController.java`: 겹치는 슬러그에 숫자 붙이기

```
// PostController: 이미 있는 슬러그면 -2, -3을 붙여 본다
private String uniqueSlug(String base) {
    String slug = base;
    int n = 2;
    while (existsSlug(slug)) {
        slug = SlugUtils.withSuffix(base, n++);
    }
    return slug;
}
```

```
private boolean existsSlug(String slug) {
    return posts.values().stream().anyMatch(p -> p.slug().equals(slug));
}
```


`주간-회고`, `주간-회고-2`, `주간-회고-3`. 이것도 좋은 방법이다. 그런데 두원이 첫 번째 "주간 회고"를 지우고 새로 쓰면 어떻게 될까? 이미 친구들에게 보낸 `주간-회고` 링크는 이제 다른 글을 가리키게 될까? 두원은 거기까지 생각하지 못했고, AI도 묻지 않았다.

증상이 보일 때마다 "고쳐 줘"를 반복하면, 블로그는 보이는 문제만 막는 덧대기 코드로 채워진다. 고칠 때마다 AI는 다른 곳을 건드리고, 두원은 무엇이 바뀌었는지 다 알지 못하며 확인할 방법도 없다. 다음 기능을 넣으면 오늘 고친 것이 다시 깨질지 모른다.

문제는 AI가 아니라 순서다. **원하는 것을 정하기 전에 고치기부터 시작했다.** 같은 증상이라도 규칙을 먼저 한 문장 적고 맡기면 결과가 달라진다.

같은 기능, 두 가지 요청

그냥 요청: "임시저장한 글이 목록에 나와. 고쳐 줘."

```
// list(): 목록에만 덧댔다
model.addAttribute("posts", posts.values().stream()
        .filter(p -> !p.draft())
        .toList());

// detail(): 그대로다. 임시저장 글도 주소로 열린다
```

정해서 요청: "규칙: 임시저장 글은 남에게 보이지 않는다. 이 규칙이 지켜져야 할 곳을 먼저 모두 찾아 알려 줘. 그다음 규칙을 `Post` 한곳에 두고, 찾은 곳이 모두 그것을 쓰게 고쳐 줘."

```
// Post: 규칙은 한곳에
public boolean isPublished() {
    return !draft;
}

// list():   .filter(Post::isPublished)
// detail(): .filter(p -> p.slug().equals(slug) && p.isPublished())
```

무엇이 달라졌나: 그냥 요청은 신고된 증상만 막았고, 같은 원인이 상세 보기에 남았다. 정해서 요청은 규칙이 지켜져야 할 곳을 AI가 먼저 찾게 하고 규칙을 한곳에 모았다. 나중에 검색이나

API가 생겨도 `isPublished()`를 쓰는지만 확인하면 된다.

2.6 AI가 바꾼 것을 읽고 나누기

두 번의 "고쳐 줘" 사이에 두원은 커밋하지 않았다. 마지막 커밋은 임시저장과 슬러그를 받아들인 직후에 해 둔 것이었다. 무엇이 바뀌었는지 보려고 `git status`를 입력하자 파일 다섯 개가 나왔다.

```
Changes not staged for commit:
        modified:   pom.xml
        modified:   src/main/java/blog/PostController.java
        modified:   src/main/java/blog/SlugUtils.java
        modified:   src/main/resources/templates/posts/list.html

Untracked files:
        src/main/java/blog/RelativeTimeFormatter.java
```

요청은 두 개였는데 바뀐 파일이 넷, 새 파일이 하나다. 두원은 `git diff`로 하나씩 열어 보았다. `PostController`에는 목록의 `filter` 한 줄과 슬러그 뒤에 숫자를 붙이는 반복문이 섞여 있었다. `SlugUtils`에는 숫자를 붙이는 메서드가 새로 생겼다. 여기까지는 요청한 것이다. 나머지는 요청한 적이 없었다. `list.html`의 날짜는 `2026-10-18`에서 "3일 전"으로 바뀌었고, `pom.xml`에는 그 표시를 위한 라이브러리가 하나 더해져 있었다. AI는 목록을 고치는 김에 목록 화면도 "개선"했다.

```
$ git diff src/main/resources/templates/posts/list.html
@@ -6,9 +6,8 @@
   <li th:each="post : ${posts}">
     <a th:href="@{/posts/{slug}(slug=${post.slug})}"
        th:text="${post.title}">제목</a>
-    <span th:text="${#temporals.format(post.createdAt, 'yyyy-MM-dd')}">
-      날짜</span>
+    <span th:text="${@relativeTime.format(post.createdAt)}">날짜</span>
   </li>
 </ul>
```

이 한 줄만 보고는 왜 바뀌었는지 알 수 없다. `@relativeTime`이 무엇인지 찾아보니, `git status` 맨 아래 "추적하지 않는 파일(Untracked files)"에 있던 `RelativeTimeFormatter`였다. 이 새

부품이 `pom.xml`에 더한 라이브러리를 쓰고 있었다. 새 파일은 Git이 아직 모르는 파일이라 `git diff`에는 나오지 않는다. `git diff`만 보고 `git status`를 건너뛰면 AI가 새로 만든 파일을 통째로 놓친다. 요청에 없던 변경은 대개 이렇게 여러 파일에 흩어져 서로 얽혀 있다. 하나만 버리면 컴파일이 실패한다.

나쁜 변경은 아니었다. 하지만 두원이 정한 변경도 아니었다. diff는 AI가 한 일의 영수증이다. AI가 일을 마치면 사람은 요청과 영수증을 나란히 놓고, 줄마다 "이건 내가 시킨 것인가"를 먼저 묻는다. 이 질문은 12장에서 리뷰 질문으로 발전한다.

성격이 다른 변경은 따로 커밋한다

다섯 파일의 변경은 성격이 셋이었다. 임시저장 글 숨기기, 같은 제목에 숫자 붙이기, 날짜 표시 바꾸기. 한 커밋에 담으면 나중에 셋 중 하나만 되돌릴 수 없다. 그런데 `PostController` 한 파일 안에 두 변경이 섞여 있었다. 이럴 때는 파일이 아니라 바뀐 덩어리 단위로 고른다.

여기서 Git의 기본 개념 하나를 짚고 간다. Git은 바뀐 것을 곧바로 기록하지 않는다. `git add`로 다음 커밋에 넣을 변경을 먼저 골라 담고, `git commit`은 담아 둔 것만 기록으로 남긴다. 이렇게 골라 두는 일을 담기(staging)라고 한다. 1장의 `git add .`는 바뀐 것을 전부 담는 명령이었고, 앞의 `git status`에 나온 "Changes not staged for commit"은 아직 담지 않은 변경이라는 뜻이다.

`git add`에 `-p`를 붙이면 Git이 바뀐 덩어리를 하나씩 보여 주며 이번 커밋에 넣을지 묻는다.

터미널

```
$ git add -p src/main/java/blog/PostController.java
@@ -18,7 +18,9 @@ public class PostController {

     @GetMapping

     public String list(Model model) {
-        model.addAttribute("posts", posts.values());
+        model.addAttribute("posts", posts.values().stream()
+                .filter(p -> !p.draft())
+                .toList());
         return "posts/list";
     }

(1/2) Stage this hunk [y,n,q,a,d,s,e,?]? y
@@ -41,6 +43,20 @@ public class PostController {
...  (uniqueSlug 반복문)
(2/2) Stage this hunk [y,n,q,a,d,s,e,?]? n
```

`y`는 이 덩어리를 담고, `n`은 건너뛴다. 덩어리가 너무 커서 그 안의 일부만 담고 싶으면 `s`로 더 잘게 쪼갠다. 다 고른 뒤에는 커밋하기 전에 무엇을 담았는지 한 번 더 본다.

터미널

```
$ git diff --staged      # 이번 커밋에 들어갈 것 (filter 덩어리만 보여야 한다)
$ git diff               # 아직 담지 않은 것 (반복문, SlugUtils, 날짜 표시)
$ git commit -m "목록에서 임시저장 글 제외 (남에게 미완성 글이 보임)"
```

두원은 `filter` 덩어리만 담아 첫 커밋을 만들고, 남은 덩어리와 `SlugUtils`로 둘째 커밋을 만들었다. 날짜 표시는 `git restore pom.xml src/main/resources/templates/posts/list.html`로 버렸다. `restore`는 Git이 아는 파일만 되돌리니, 새 파일 `RelativeTimeFormatter.java`는 직접 지웠다. 필요하면 나중에 따로 맡기면 된다. `git log --oneline`을 입력하자 무엇을 왜 했는지가 한 줄씩 보였다.

터미널

```
$ git log --oneline
c41d2e9 같은 제목이면 슬러그 뒤에 숫자 붙이기
7a9f310 목록에서 임시저장 글 제외 (남에게 미완성 글이 보임)
5be02c4 임시저장과 제목 주소 추가
1f3a8d0 v0: 한 줄 요청으로 만든 블로그
```

맨 위가 가장 최근 커밋이다. 앞의 일곱 글자는 커밋마다 붙는 고유 번호(해시)의 앞부분이다. 되돌리거나 비교할 때 이 번호로 커밋을 가리킨다.

2.7 되돌리고, 브랜치에서 시도하기

다음 날 두원은 2.5절에서 품었던 의문으로 돌아왔다. 첫 "주간 회고"를 지우고 새로 쓰면 `주간-회고` 주소는 새 글을 가리킨다. 숫자를 붙이는 방법이 틀렸다기보다, 지운 뒤에 어떻게 할지 정하지 않은 채로는 받아들일 수 없었다. 되돌리는 방법을 AI에게 물었더니 `git reset --hard HEAD~1`이라는 답이 돌아왔다.

명령을 입력하기 직전에 두원은 멈칫했다. 그사이 상세 보기에서도 임시저장 글을 숨기도록 고친 코드가 아직 커밋하지 않은 채 남아 있었다. Git으로 되돌리는 방법은 두 가지다.

- `git revert 커밋번호`는 그 커밋이 한 일을 거꾸로 하는 **새 커밋**을 만든다. 원래 커밋도, 되돌렸다는 사실도 기록에 남는다. 이미 GitHub에 올린 커밋은 이쪽으로 되돌린다.
- `git reset`은 브랜치가 가리키는 곳을 **뒤로 옮긴다.** 그 뒤의 커밋은 기록에서 사라진다. `--hard`를 붙이면 커밋하지 않은 작업까지 지운다. 아직 내 컴퓨터에만 있는 커밋을 없던 일로 할 때만, 무엇이 사라지는지 알고 쓴다.

AI가 알려 준 대로 실행했다면 숫자 붙이기와 함께 상세 보기 수정도 사라졌을 것이다. AI는 작업 폴더에 무엇이 남아 있는지 묻지 않았고, 두원도 말하지 않았다. 두원은 숫자 붙이기 커밋을 어젯밤 GitHub에 올려 두었다는 것을 확인하고 `git revert`를 골랐다.

터미널

```
$ git revert c41d2e9
error: your local changes would be overwritten by revert.
hint: commit your changes or stash them to proceed.
fatal: revert failed
```

Git이 먼저 멈췄다. 되돌릴 커밋과 아직 커밋하지 않은 상세 보기 수정이 같은 `PostController`를 건드리니, 그대로 하면 수정이 덮인다는 뜻이다. 커밋하지 않은 변경은 `git stash`로 잠시 치워 둘 수 있다.

터미널

```
$ git stash              # 커밋하지 않은 변경을 임시 보관함에 넣는다
$ git revert c41d2e9     # 편집기가 열리면 메시지를 확인하고 저장한다
$ git stash pop          # 보관해 둔 변경을 다시 꺼낸다
$ git log --oneline -3
e08b7a1 Revert "같은 제목이면 슬러그 뒤에 숫자 붙이기"
c41d2e9 같은 제목이면 슬러그 뒤에 숫자 붙이기
7a9f310 목록에서 임시저장 글 제외 (남에게 미완성 글이 보임)
```

숫자 붙이기는 사라졌지만, 붙였다가 되돌렸다는 사실은 기록에 남았다. 이미 이 커밋을 받아 간 사람이 있어도 그 사람의 기록과 어긋나지 않는다. 다음에 `git push`를 하면 되돌린 커밋이 그대로 올라간다. 꺼내 둔 상세 보기 수정은 이 장이 끝날 때까지 커밋하지 않았다. 그래서 예제 저장소 `v1` 브랜치에도 들어 있지 않다. 임시저장 글은 목록에서만 빠지고, 주소를 알면 상세 화면에서 열린다.

`reset`도 알아 두자. `git reset HEAD~1`의 `HEAD~1`은 "지금보다 하나 앞 커밋"이다. 옵션 없이 쓰면 마지막 커밋만 취소하고, 바뀐 코드는 담기 전 상태로 작업 폴더에 남긴다. `--soft`를 붙이면 담아 둔 상태까지 그대로 남긴다. 둘 다 코드는 그대로이니, 커밋 메시지를 잘못 썼거나 커밋을 다시 나누고 싶을 때 쓸 만하다. `--hard`만 다르다. 커밋하지 않은 작업 폴더의 변경까지 **지운다**. `--hard`로 잃어버린 커밋은 `git reflog`에서 번호를 찾아 되살릴 수 있다. Git은 브랜치가 거쳐 간 자리를 한동안 기억한다. 하지만 한 번도 커밋하지 않은 변경은 Git도 모른다. 그래서 되살릴 길이 없다.

이때부터 두원은 규칙 하나를 정해 두었다. AI가 Git 명령을 알려 주거나 직접 실행하겠다고 할 때, 지우거나 기록을 고치는 명령(`reset --hard`, `push --force`, `clean`)은 실행하기 전에 무엇이 사라지는지 AI에게 먼저 말하게 한다. 실행은 사람이 한다. 15장에서는 이런 명령을 AI 도구가 아예 실행하지 못하게 막는 방법을 본다.

AI에게 넘기면 안 되는 정보

되돌린 커밋을 올리려고 `git push`를 입력하자 인증에 실패했다는 오류가 났다. 준비물로 토큰을 만들 때 유효 기간을 7일로 골라 두었는데, 그 7일이 지났다. 두원은 GitHub에서 토큰을 새로 만들고 오류 메시지와 함께 AI 채팅에 붙여 넣었다.

push가 안 돼. 오류는 아래와 같아. 토큰은 새로 만들었어. ghp_7Qm2xK…(전체 값) 다시는 안 묻게 해 줘.

AI는 친절했다. 토큰을 담은 `.env`와, 그 값을 읽어 push하는 `push.sh`를 프로젝트 폴더에 만들었다. 2.6절에서 배운 대로 `git status`를 입력하자 두 파일이 보였다.

터미널

```
$ git status --short
?? .env
?? push.sh
```

습관대로 `git add .`를 입력하려다 두원은 그만두었다. 이 한 줄이면 토큰이 커밋에 들어가고, 다음 push로 GitHub에 올라간다.

옆에서 보던 마르코가 고개를 저었다.

> **마르코** 커밋 안 했으니 됐다고 생각하지? 그 토큰은 이미 채팅창에 붙여 넣었잖아.

AI에게 건넨 내용은 내 컴퓨터를 떠난다. 서비스의 설정과 회사가 맺은 계약에 따라 서비스 쪽에 저장되거나 기록에 남을 수 있다. 그래서 비밀값은 붙여 넣는 순간, 또는 커밋하는 순간 샌 것으로 본다. 대화에서 지우거나 마지막 커밋에서 빼도 되돌릴 수 없다. Git 기록에는 옛 커밋이 그대로 남는다. 할 수 있는 일은 그 값을 폐기하고 새로 발급하는 것뿐이다. 커밋하지 않았다고 해서 AI에게 넘어가지 않은 것도 아니다. 작업 폴더를 읽는 AI 도구는 Git이 추적하지 않는 파일도, `.gitignore`로 무시한 파일도 열 수 있다.

무엇을 넘겨도 되는지는 세 칸으로 판단한다.

표 2-2 AI에게 넘겨도 되는지 판단하는 세 칸

칸	무엇	어떻게
넘기지 않는다	비밀번호, API 키, 토큰, 개인 키 같은 비밀값. 운영 데이터의 개인정보	어떤 형태로도 붙여 넣지 않는다
가리고 줄여서 넘긴다	로그, 스택 트레이스, 예시 데이터, 설정 파일	이메일과 ID는 가짜 값으로 바꾸고, 필요한 줄만 자른다
그대로 넘긴다	비밀값이 없는 내 코드, 공개 문서	그대로 넘긴다

블로그에 데이터베이스와 독자가 생기면 데이터베이스 비밀번호와 독자의 이메일이 첫 칸에 더해진다. 회사에서 일한다면 어떤 AI 서비스를 어떤 설정으로 써도 되는지 정한 규칙부터 확인한다.

도움을 청하는 데 비밀값이 필요한 경우는 드물다. 두원이 원한 것은 "토큰을 다시 묻지 않게" 였고, 그것은 토큰 값 없이도 물을 수 있었다.

```
git push가 인증 실패로 멈췄어. 오류 메시지는 아래와 같아.
토큰을 매번 넣지 않을 방법을 알려 줘.
토큰 값은 묻지 마. 프로젝트 폴더 안에 토큰을 담은 파일을 만들지 마.
방법마다 토큰이 어디에 저장되는지 함께 적어 줘.
```

값이 필요해 보이는 문제도 대개 모양만 있으면 된다. 오류는 가장 작은 재현으로 줄이고, 로그는 필요한 줄만 자르고, 데이터는 붙여 넣는 대신 "이메일 칸이 빈 행이 있다"처럼 모양을 말한다. 보내기 전에는 세 가지를 본다. 비밀번호, 키, 토큰이 든 줄이 없는가? 실제 사람의 이메일이나 이름이 `reader1@example.com` 같은 가짜 값으로 바뀌었는가? 문제를 보이는 데 필요한 줄만 남았는가? 받은 결과에서는 이번처럼 AI가 값을 파일에 적지 않았는지 본다. "다시 묻지 않게"를 가장 빨리 해결하는 방법이라 AI가 자주 고른다. diff와 `git status`에 `.env`, 키 파일, 값이 직접 적힌 설정이 새로 생기지 않았는지 확인한다.

말로 부탁해 두면 다음번에는 잊는다. 그래서 정한 것을 장치로 옮긴다. 로컬 설정과 `.env`는 `.gitignore`에 넣는다. 비밀값은 파일에 쓰지 않고, 설정에는 환경 변수 이름만 둔다(15장). 커밋하기 전에 비밀값처럼 생긴 문자열을 찾는 검사를 둔다. AI에게는 "비밀값 파일(`.env` 등)은

읽거나 출력하지 않는다. 값이 필요하면 묻는다"는 규칙을 준다. 지금은 모니터 옆에 붙여 두고, 15장에서 AI context 문서를 만들 때 옮긴다.

두원의 블로그에서는 이렇게 적용했다. 먼저 GitHub에서 붙여 넣은 토큰을 폐기했다. 새 토큰은 이 저장소 하나에만 쓰게 만들고, Git의 자격 증명 도우미가 운영체제의 저장소에 보관하게 했다. 폴더의 `.env`와 `push.sh`는 지웠다. 그다음 `.gitignore` 끝에 `.env`, `.env.*`, `!.env.example` 세 줄을 더하고, 혹시 예전에 커밋한 적은 없는지 기록 전체에서 토큰 앞부분을 찾았다.

터미널

```
$ git check-ignore -v .env
.gitignore:20:.env  .env
$ git log -p -S 'ghp_7Qm2xK' --all
$ git grep 'ghp_7Qm2xK' $(git rev-list --all)
```

`check-ignore`는 어느 줄이 그 파일을 무시하게 하는지 알려 준다. 그 뒤의 두 명령은 모든 커밋의 변경과 내용에서 그 문자열을 찾는다. 아무것도 나오지 않았으니 기록에는 없다. 마지막으로 커밋 전에 실행되는 훅을 저장소에 두었다.

예제 2-5 scripts/hooks/pre-commit: 비밀값 모양을 막는 커밋 전 훅

```
#!/bin/sh
# scripts/hooks/pre-commit: 더하는 줄에 비밀값 모양이 있으면 멈춘다
P='ghp_[A-Za-z0-9]{20,}|github_pat_[A-Za-z0-9_]{20,}|PRIVATE KEY-{5}'
if git diff --cached -U0 | grep '^+' | grep -qE "$P"; then
  echo "비밀값으로 보이는 줄이 있다. 빼고 다시 담는다." >&2
  exit 1
fi
```

저장소에서 아래 명령을 한 번 실행하면 이 폴더의 훅이 커밋 전에 실행된다.

터미널

```
$ git config core.hooksPath scripts/hooks
```

토큰이 든 파일을 담아 커밋해 보니 훅이 커밋을 막았다.

터미널

```
$ git commit -m "메모"
비밀값으로 보이는 줄이 있다. 빼고 다시 담는다.
```

훅은 `--no-verify`로 건너뛸 수 있고 클론마다 설정해야 한다. 15장에서는 AI가 이 옵션을 쓰지 못하게 막는다. 다른 언어와 서비스를 쓰더라도 할 일은 같다. 세 칸으로 판단하고, 무시 목록, 환경 변수, 커밋 전 검사, AI 규칙을 내 환경의 도구로 옮긴다.

시도는 브랜치에서

숫자 붙이기를 되돌리자 같은 제목 문제가 다시 남았다. 방법은 여럿 떠올랐다. 날짜를 붙일까, 글 번호를 함께 쓸까. 하나씩 AI에게 맡겨 보고 싶었지만, 그때마다 `main`을 어지럽히고 싶지는 않았다. 두원은 시도마다 브랜치를 땄다.

터미널

```
$ git switch -c try/slug-date      # main에서 갈라져 나온 시도용 브랜치
  # AI에게 맡기고, 확인하고, 마음에 들지 않으면
$ git switch main
$ git branch -D try/slug-date      # 브랜치를 통째로 버린다
```

시도용 브랜치에서 AI가 일을 마치면, 커밋한 뒤 `main`과 얼마나 달라졌는지부터 본다.

터미널

```
$ git branch                  # 브랜치 목록. * 표시가 지금 있는 곳이다
$ git diff main --stat        # main과 비교해 바뀐 파일과 줄 수
```

브랜치는 만들고 버리는 데 1초면 된다. 날짜를 붙이는 시도는 하루에 같은 제목으로 두 번 쓰면 다시 겹쳤다. 글 번호를 붙이는 시도는 1장의 숫자 주소로 돌아간 것 같았다. 두 브랜치 모두 버렸고, 그동안 `main`은 한 번도 흔들리지 않았다.

그런데 브랜치를 버리면서 두원은 이상한 점을 느꼈다. AI는 시도마다 잘 만들었고, 두원은 매번 "이건 아닌데"라고 판단했다. 그 판단의 기준은 어디에도 적혀 있지 않았다.

이 일을 겪은 뒤 두원은 일을 맡기는 문장부터 바꿨다. "같은 제목으로 글을 쓰면 하나만 보여. 고쳐 줘." 대신 이제는 이렇게 맡긴다.

같은 제목으로 글을 쓰면 하나만 보여. 고칠 방법을 두세 가지 먼저 알려 줘. 내가 하나를 고르면 그것만 고쳐. 고치기 전에 바꿀 파일과 이유를 보여 주고, 요청과 상관없는 파일은 건드리지 마.

AI는 코드를 쓰지 않고 이렇게 답했다.

방법은 네 가지가 있습니다.

1. 뒤에 숫자 붙이기(`주간-회고-2`): 읽기 쉽습니다. 다만 앞의 글을 지우면 그 주소를 새 글이 가져갈 수 있습니다.
2. 앞에 날짜 붙이기(`2026-10-20-주간-회고`): 언제 쓴 글인지 보입니다. 같은 날 같은 제목이면 다시 겹칩니다.
3. 글 번호 함께 쓰기(`/posts/12/주간-회고`): 겹치지 않습니다. 주소에 숫자가 다시 들어갑니다.
4. 짧은 무작위 문자열 붙이기(`주간-회고-x7k2`): 겹칠 일이 거의 없습니다. 주소를 기억하기 어렵습니다.

바꿀 파일은 어느 방법이든 `PostController`와 `SlugUtils` 두 개입니다. 어느 것이 맞는지는 지운 글의 주소를 어떻게 할지, 주소에 날짜나 숫자가 보여도 되는지에 달려 있습니다. 어떻게 할까요?

두원이 브랜치에서 하나씩 해 보며 겪은 일이 선택지마다 한 줄씩 적혀 있었다. 그리고 마지막 문장이 두원을 멈추게 했다. 고르려면 먼저 정해야 할 것이 있었다. 지운 글의 주소는 어떻게 되는가? 주소에 날짜나 숫자가 보여도 되는가? 두원은 아직 어느 쪽도 정하지 않았다. 브랜치를 두 번 버린 이유가 그것이었다.

두 번의 "고쳐 줘"가 남긴 것은 성격이 다른 변경 셋이 섞인 파일 다섯 개였다. 새 요청은 AI가 선택지부터 내놓게 하고, 사람이 고른 뒤에야 작은 diff를 만들게 한다. 그 diff는 읽을 수 있고, 커밋 하나에 담기며, 되돌려도 다른 것이 딸려 가지 않는다.

두원은 모니터 옆에 네 줄을 붙여 두었다.

1. 맡기기 전에 커밋한다.
2. 맡긴 뒤에는 `git status`와 `git diff`로 요청에 없던 변경을 찾는다.
3. 성격이 다른 변경은 따로 커밋하고, 메시지에 무엇을 왜 바꿨는지 쓴다.
4. 시도는 브랜치에서 하고, 아니면 버린다. 이미 올린 커밋은 `revert`로 되돌린다.

2.8 개발자의 일이 바뀐다

버린 브랜치 이야기를 들은 마르코는 두원에게 종이 한 장을 내밀었다.

> **마르코** 고생했어. 그럼 이번엔 코드 말고 이것부터 적어 볼래? 임시저장 글은 누가 볼 수 있어야 해? 같은 제목의 글 주소는 어떻게 달라야 해? 글을 지우면 그 주소는 어떻게 돼?

코드를 직접 쓰던 때에는 개발자의 시간 대부분이 코드를 쓰는 데 들었다. 무엇을 만들지 정하는 일은 코드를 쓰면서 자연스럽게 함께 했다. 직접 `if`문을 쓰다 보면 "임시저장 글이면 어떻게 하지?"라는 질문과 저절로 마주쳤기 때문이다. 앞 권에서 두원이 `ORDER BY` 한 줄을 쓰며 목록의 순서를 정한 것도 그런 마주침이었다.

AI가 코드를 써 주면서 그 마주침이 사라졌다. 코드는 순식간에 생기고, 질문은 아무도 던지지 않은 채 지나간다. 그래서 이제는 그 질문을 일부러 던져야 한다. 개발자의 일이 어디로 옮겨 가는지는 그림 2-3과 같다.

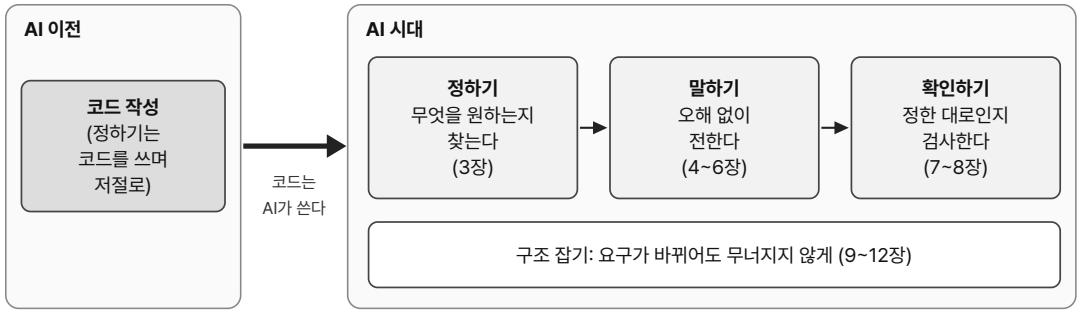


그림 2-3 개발자의 일의 이동: 코드 작성에서, 정하기·말하기·확인하기로

AI 시대 개발자의 일은 세 가지로 정리할 수 있다.

1. **정하기:** 내가 무엇을 원하는지 스스로 찾아낸다. (3장)
2. **말하기:** 정한 것을 AI가 오해하지 않게 전한다. (4~6장)
3. **확인하기:** 만들어진 것이 정한 대로인지 검사한다. (7~8장)

그리고 요구가 계속 바뀌어도 이 세 가지가 무너지지 않게 구조를 잡는 일이 그 위에 놓인다(9~12장). 그 뒤의 장들은 같은 세 가지를 팀으로, 진짜 사용자 앞으로, 오래 운영되는 블로그로 넓혀 갈 뿐이다. 일의 이름은 바뀌지 않는다.

두원은 마르코의 종이를 다시 보았다. 세 질문 모두 첫 번째 일, 정하기에 속했다. 그런데 막상 답을 적으려니 펜이 움직이지 않았다.

개발 상식 브랜치 전략과 커밋 메시지

2.6절과 2.7절에서 두원은 diff를 읽고, 변경을 나눠 커밋하고, `revert`로 되돌리고, 시도용 브랜치를 버렸다. 여기서는 그 습관을 팀의 방식으로 넓혀 본다.

기능 하나에 브랜치 하나. 이 책은 GitHub Flow라는 단순한 방식을 쓴다. `main`은 늘 동작하는 상태로 두고, 기능마다 브랜치를 따서 작업한다. 다 되면 풀 리퀘스트(PR)로 바뀐 내용을 확인하고 `main`에 합친다. 시도용 브랜치와 다른 점은 버릴 생각으로 따지 않는다는 것뿐이다. 한 번의 흐름은 이렇다.

터미널

```
$ git switch -c feature/draft-visibility    # main에서 갈라진다
  # AI에게 맡기고, diff를 읽고, 나눠 커밋한다
$ git push -u origin feature/draft-visibility
  # GitHub에서 PR을 열고, 바뀐 내용을 확인한 뒤 main에 합친다
$ git switch main
$ git pull                                  # 합쳐진 main을 받아 온다
$ git branch -d feature/draft-visibility    # 다 쓴 브랜치를 지운다
```

`-d`는 이미 합쳐진 브랜치만 지운다. 합치지 않은 브랜치는 "아직 합치지 않았다"며 거절한다. 2.7절에서 시도용 브랜치를 지울 때 `-D`를 쓴 것은, 합치지 않은 줄 알고 일부러 버렸기 때문이다.

혼자 하는 블로그에도 PR을 쓰는 이유가 있다. PR 화면은 브랜치의 모든 변경을 한곳에 모아 보여 주고, 줄마다 메모를 남길 수 있다. AI의 변경을 영수증처럼 읽기에 좋은 자리다. 8장부터는 이 자리에 테스트와 품질 검사가 붙어, 사람이 보기 전에 기계가 먼저 본다. 브랜치를 쓰는 두 방식을 견주면 그림 2-4와 같다.

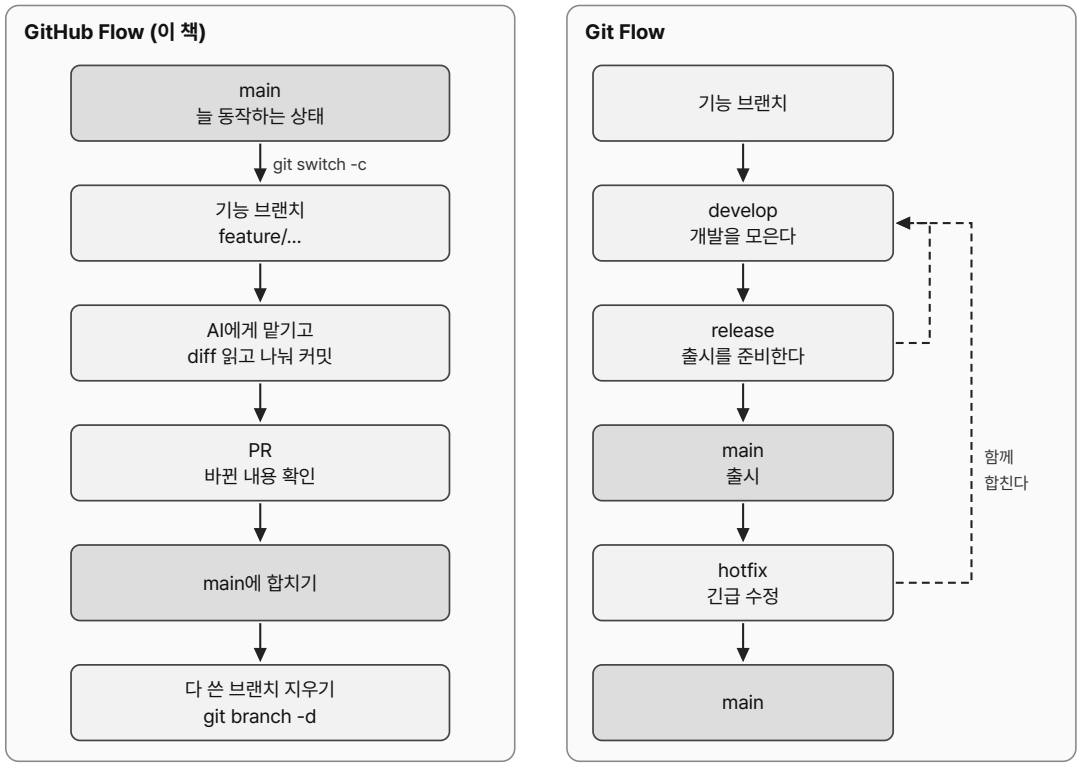


그림 2-4 GitHub Flow와 Git Flow: 기능 브랜치 하나를 main에 합치는 흐름과, develop·release·hotfix를 거치는 흐름

브랜치를 더 나누는 Git Flow도 있다. `main` 말고도 개발을 모으는 `develop`, 출시를 준비하는 `release`, 긴급 수정용 `hotfix` 브랜치를 두고 정해진 순서로 합친다. 출시 일정이 정해진 제품에 잘 맞지만, 작은 팀이 자주 배포하는 블로그에는 GitHub Flow가 단순하다.

커밋 메시지는 "무엇을, 왜". "수정"이나 "AI 작업" 같은 메시지는 나중에 아무 도움이 안 된다. "임시저장 글을 목록에서 제외 (남에게 미완성 글이 보임)"처럼 무엇을 왜 바꿨는지 쓴다. 변경을 성격별로 나눠 커밋해 두면 메시지도 한 줄로 쓸 수 있다.

이 장에서 배운 것

항목	핵심
말한 것과 원한 것	AI는 요청에 들어 있는 것을 정확히 만들고, 요청에 없던 것은 우연한 동작으로 채운다. 문제는 AI의 실력이 아니라 원한 것과 말한 것 사이의 틈에 있고, 그 틈은 사람 쪽에 있다.
"고쳐 줘"의 함정	증상마다 "고쳐 줘"를 반복하면 덧대기 코드만 늘어난다. 고치기 전에 원하는 규칙을 먼저 정하고, 그 규칙이 지켜져야 할 곳을 모두 찾게 한다.
diff 읽기	맡기기 전에 작업 폴더를 깨끗하게 하고, AI가 일을 마치면 diff를 영수증처럼 읽는다. 바뀌었어야 하는데 바뀌지 않은 곳은 diff에 보이지 않는다.
나누고 되돌리기	성격이 다른 변경은 `git add -p`로 나눠 커밋한다. 올린 커밋은 `revert`로, 시도는 브랜치째 버리는 것으로 되돌린다. 지우는 명령은 무엇이 사라지는지 확인한 뒤 사람이 입력한다.
넘기지 않을 정보	비밀값은 AI에게 붙여 넣거나 커밋하는 순간 유출된 것으로 보고 폐기한다. 넘길 것은 세 칸(넘기지 않는다, 가리고 줄여서, 그대로)으로 판단하고, 무시 목록과 커밋 전 검사로 막는다.
개발자의 일	AI와 개발할 때 개발자의 일은 정하기, 말하기, 확인하기다.

이 장의 코드는 예제 저장소 `v1` 브랜치에 있다(부록 E). 저장소에는 `SlugUtils`의 동작(공백은 하이픈으로, 앞뒤 공백은 지우고, 영문은 소문자로)을 확인하는 테스트 세 개를 덧붙여 두었다. 테스트를 쓰는 법은 7장에서 다룬다.

연습 문제

1. 자신의 AI 도구에 2.1절의 요청을 그대로 넣어 보자. 2.3절의 세 가지 문제 중 몇 개가 나타나는가? 도구에 따라 일부는 AI가 미리 막아 줄 수도 있다. 그렇다면 AI가 대신 내린 결정이 무엇인지 적어 보자.
2. 표 2-1에 "두원이 원했지만 말하지 않은 것"을 세 개 더 찾아 적어 보자. (힌트: 임시저장 글을 발행하면? 제목을 바꾸면 주소는?)

3. 2.3절의 `curl` 명령으로 세 가지 문제를 자신의 블로그에서 재현해 보자. 고친 뒤 같은 명령을 다시 실행해 정말 사라졌는지 확인하자.
4. AI에게 일을 맡기기 전후로 `git diff`를 실행해, 요청과 상관없이 바뀐 코드가 있는지 찾아보자. 있다면 `git add -p`로 요청한 변경만 골라 커밋하고 나머지는 `git restore`로 버려 보자.
5. 시도용 브랜치를 하나 따서 같은 기능을 다른 방법으로 AI에게 맡긴 뒤, 브랜치를 버리고 `main`이 그대로인지 확인해 보자.
6. 내 프로젝트 폴더에서 아래 첫 명령을 실행해, 커밋하지 않았지만 AI 도구가 읽을 수 있는 파일 중 비밀값이 든 것이 있는지 찾아보자. 있다면 둘째 명령에 그 파일 이름을 넣어 `.gitignore`에 들어 있는지 확인하자.

터미널

```
$ git status --short --ignored
$ git check-ignore -v <파일 이름>
```

마르코의 질문

상황: 운영 중인 블로그에 버그 신고가 들어오자 동료가 메신저로 묻는다.

"AI한테 고쳐 달라고 하면 5분이면 끝나지 않아요?"

짚어야 할 것

- 고치는 데는 5분이면 된다. 하지만 증상만 고치면 같은 원인이 다른 곳에 남는다. 임시저장 글이 목록에서는 사라졌지만 주소로는 열렸던 것처럼.
- 먼저 원래 원했던 규칙을 한 문장으로 적는다. "임시저장 글은 작성자만 볼 수 있다."
- 그 규칙이 지켜져야 하는 모든 곳(목록, 상세, 검색, API)을 확인하고, 확인을 테스트로 남긴 뒤에 수정을 맡긴다.

빠지기 쉬운 답: "네, 금방 돼요." 그렇게 시작한 수정은 대개 두더지 잡기로 이어진다.

두원 적어 보려는데 잘 안 써져. 내가 뭘 원하는지 나도 잘 모르겠어.

마르코 당연하지. 질문을 받기 전에는 아무도 몰라. 그러니까 질문을 받아 봐.

두원 누구한테?

마르코 매일 코드 작성해 달라고 시키는 그 친구한테. 이번엔 코드 말고 질문을 달라고 해 봐.